MDPI

Article

Scalable Extraction of Author-Stated Research Gaps: Towards Automated Idea Generation

Yazan Alnakri [1,*], Osama Orabi [1,2] and Hamza Shafee Aldaghstany [1]

[1] Laboratory for Robotics and Artificial Intelligence, Innopolis University, 420500 Innopolis, Russia; y.alnakri@innopolis.university (Y.A.); o.orabi@innopolis.university (O.O.); h.aldaghstany@innopolis.university (H.S.A.)
[2] Moscow Independent Research Institute of Artificial Intelligence (MIRAI), 117218 Moscow, Russia
* Correspondence: y.alnakri@innopolis.university

Abstract

Papers state their own research gaps: limitations, future work and open problems. These statements could seed automated idea generation, but they are rare, scattered and easily confused with other text. Asking a large language model (LLM) to judge every sentence is costly, and giving it the whole paper buries the few relevant sentences. We instead narrow each paper with a funnel: rules discard unlikely text, a natural-language-inference classifier proposes candidates, and the LLM only verifies them. Every output is a sentence the authors wrote, and one instruction sets the verifier to favour precision or recall. We evaluate on human-labelled limitation (SAL) and future-work (FWS) benchmarks and on 230 LLM-labelled papers not yet checked by people. The instruction controlled the trade-off on every dataset, and verification more than doubled SAL precision at 85% recall. In its cheapest variant (one call per paper, classifier fine-tuned on our LLM labels), the funnel reads 7–8 times fewer input tokens than whole-paper prompting and matches or improves F_1 on both human-labelled benchmarks at every model size. Small models gain most: with a 4B LLM, F_1 on arXiv rose from 0.30 to 0.48. With a 27B LLM, whole-paper prompting stayed ahead on arXiv.

Keywords: research gap extraction; limitations; future work; scientific text mining; large language models; natural language inference; zero-shot classification; LLM filtering

1. Introduction

Research papers often say where a study falls short and what should be done next. Authors acknowledge the limitations of their own work, which tell readers how far its design, execution and interpretation limit the findings [1,2]. They also propose future work, naming the extensions and open questions they consider worth pursuing [3,4], and they point to unsolved problems in their field when they motivate their own work. We call these statements *author-stated research gaps*. Collected across many papers, they are a natural input for automated idea generation, because each one is a domain expert's claim about what is still missing.

Gap statements are hard to find for three reasons. First, they are rare. In the test sets we use, they make up only 2.6% of the sentences in randomized controlled trial (RCT) reports, 5.7% in arXiv and biomedical papers and 14.6% in conclusion sections. Second, they are scattered. A limitation may sit in a dedicated section, in the discussion or next to a methodological choice, and in computer-science papers many gaps appear in the methods

Received:
Revised:
Accepted:
Published:

Copyright: © 2026 by the authors. Submitted to *Mach. Learn. Knowl. Extr.* for possible open access publication under the terms and conditions of the Creative Commons Attribution (CC BY) license.

and experiments sections. Third, many sentences look like gaps without being one, such as a reported result or a piece of background that mentions a difficulty. One apparent shortcut does not solve the problem either. Large language models (LLMs) can be prompted to write limitations and critiques of a paper [5,6], but such text expresses the model's view, not the authors'. We want every output to be a sentence the authors actually wrote.

An LLM can also judge whether a given sentence states a gap, so the open question is how much of the paper to give it. Judging every sentence in a separate request is expensive. Giving the model the whole paper is simpler, but the few relevant sentences then sit among thousands to tens of thousands of tokens. LLMs use information in the middle of long inputs less reliably [7], and their accuracy falls as inputs grow [8]. This matters most for the smaller models that make large collections affordable to process.

Our idea is to make the LLM's job as small as possible. Instead of asking it to find the gaps in a paper, we ask it only to check a short list of candidates that cheaper components have already selected. Earlier extraction work points in the same direction. Clinical limitation recognition selects candidate sentences with section titles and limitation keywords, and these rules alone were slightly more accurate than supervised classifiers [1]. Future-work recognition, in turn, separates locating sentences from grouping or classifying them [3,4].

We build this division of labour into a three-stage funnel (Figure 1). Stage A removes text that is unlikely to contain gaps. Stage B selects candidate sentences with a zero-shot natural-language-inference (NLI) classifier. Stage C asks an LLM to keep or drop each candidate. This design has three useful properties. Because no stage rewrites text, every output is a sentence the authors wrote. Because Stage B needs no gap-labelled training data, the same funnel runs on biomedical and computer-science papers. And because the LLM only verifies, a single instruction can steer it: one paragraph of the system prompt, which we call the mode paragraph, sets Stage C to high precision, balanced or high recall. A false gap wastes effort in idea generation, so precision is our primary goal and high precision is the main mode.

This design raises three questions, which structure the rest of the paper.

- **RQ1. How do rules, a classifier and an LLM each contribute when they extract research gaps together?** Because the LLM can only verify what the earlier stages pass on, we measure how much precision each stage adds and how much recall it costs. We also test whether a training-free NLI classifier is good enough for Stage B, compared with a classifier trained on human labels and one fine-tuned on our LLM labels, and whether the mode paragraph alone can make the LLM stage stricter or more lenient.
- **RQ2. How does filtering a paper before an LLM reads it compare with giving the LLM the whole paper, in accuracy and cost?** This question tests our central idea. With the same LLM on both sides, we compare F_1, input tokens and time for three model sizes, sending the funnel's candidates either 20 per call or all of a paper's candidates in one call.
- **RQ3. How far can labels made by LLMs stand in for human labels when evaluating gap extraction?** Human labels exist for only one gap type per dataset, so part of our evaluation relies on labels produced by LLMs. We answer this question on the datasets that have human labels, by scoring the same SAL output against both label sets and by testing a classifier fine-tuned on LLM labels against the human labels of SAL and FWS. The answer is therefore partial, and a human study of all three gap types is the next step.

To answer these questions, we evaluate the funnel on two benchmarks with human labels: self-acknowledged limitations in RCT reports (SAL) [2] and future-work sentences in natural language processing (NLP) papers (FWS) [4]. Because each covers only one gap type in one field, we also built a corpus of 230 arXiv and biomedical papers, in which

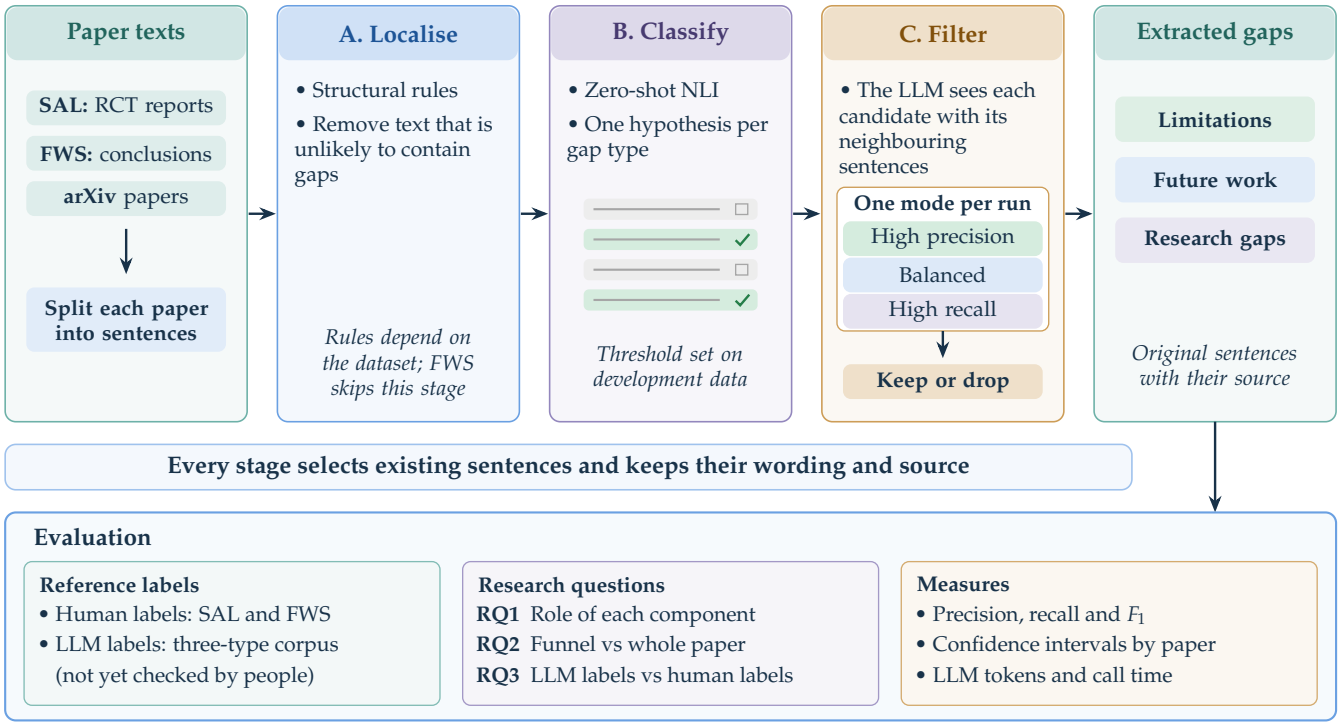


Figure 1. Overview of the funnel and its evaluation. Stage A applies rules that depend on the dataset, and FWS skips it. The three modes are alternative instructions for Stage C, not consecutive steps. SAL has human labels for limitations and FWS for future work; the three-type corpus has LLM labels, which are provisional.

two LLMs labelled all three gap types independently and an LLM judge resolved their disagreements. On these data we compare the funnel with whole-paper prompting, using three Qwen models with 27B, 9B and 4B parameters, and with published sentence-level methods that we re-ran on the same test sentences.

The main result is that narrowing the LLM's job pays off most where the LLM is weakest. With the 4B model, whole-paper prompting kept most sentences of several long arXiv papers, whereas the funnel raised F_1 on arXiv from 0.30 to 0.48 while the LLM read about seven times fewer input tokens. This variant of the funnel sends each paper's candidates in a single call and uses a Stage B classifier fine-tuned on our LLM labels (Tables 9 and 10). With the 27B model the two approaches are closer: the funnel scores higher on the human-labelled benchmarks with less input, while whole-paper prompting scores higher on arXiv papers, whose labels were themselves made from whole papers. Our contributions are as follows.

1. **A source-preserving funnel for gap extraction**, in which the LLM only verifies candidates and one instruction sets the trade-off between precision and recall. The modes kept their order on every dataset, and in high-precision mode verification raised precision on SAL from 29.1% to 66.2% at 84.9% recall (RQ1, Sections 4.1 and 4.3).
2. **A comparison with whole-paper prompting across three LLM sizes**, in both accuracy and cost. In its one-call-per-paper format, the funnel reads about 7–8 times fewer input tokens on full papers, and its advantage grows as the model gets smaller (RQ2, Sections 4.5 and 4.6).
3. **An analysis of the stages, the classifier and published baselines.** Stage A helps where section headings are informative, and Stage B sets the upper limit on recall because Stage C can only remove candidates. Stage C then supplies most of the precision gain on full papers. A zero-shot Stage B classifier comes within 0.021 F_1 of a supervised one on SAL, although it is less precise. Against published sentence-

level methods, the funnel is clearly ahead on SAL, whereas on FWS a simple trained classifier comes close to it (RQ1, Sections 4.1, 4.2 and 4.4).

4. **A three-type evaluation corpus and evidence on how far LLM labels can replace human ones.** We release the LLM-labelled corpus together with every LLM verdict behind our results. The same SAL output reaches 66.2% precision against human labels but 93.2% against LLM labels, so precision measured against LLM labels cannot be read as precision against human judgement. We also find that letter case reveals the label in the public FWS corpus, which is why we lowercase all FWS text (RQ3, Section 4.7).

The scope of this paper is extraction. Whether an extracted gap is still open, and whether it leads to a useful research idea, are questions we leave for later work. The rest of the paper is organised as follows. Section 2 reviews related work, and Section 3 describes the data, the funnel and the comparisons. Section 4 reports the results, Section 5 discusses them together with the limitations of the study, and Section 6 concludes.

2. Related Work

Our work builds on three lines of research: scientific discourse classification, the recognition of author-stated limitations and future work, and LLM-based analysis of scholarly documents. Across these lines, similar terms can refer to different tasks: locating an existing statement, inferring a missing piece of knowledge, or generating a new criticism or research direction. We organise the discussion around these distinctions and explain how each line of work informs our funnel. Table 1 compares the closest studies by target, method and input unit. Part of the literature search was supported by a tool, as described in Section 3.7.

2.1. Scientific discourse and sentence classification

Scientific discourse annotation provides a foundation for identifying statements by their role within a paper. Argumentative zoning represents the rhetorical function of sentences and their relationship to the authors' argument [9]. Liakata et al. [10] distinguish this rhetorical perspective from Core Scientific Concepts (CoreSC), which represent components of an investigation, including its goals, methods, observations and conclusions. The Dr. Inventor corpus brings the discussion closer to our targets: its 40 computer-graphics articles have a Future Work category in the argumentative-structure layer and a Limitation category among the crosswise features [11]. These annotations establish a full-text precedent for representing both kinds of statement, although they do not constitute a single research-gap detection task.

Abstract sentence classification supplies further resources and modelling approaches. PubMed 200k RCT and CSAbstruct support the recognition of abstract roles in biomedical and computer-science publications, respectively [12,13]. MAZEA allows multiple rhetorical labels per sentence [14], while Emerald 110k extends coverage across disciplines and includes a combined Research limitations/implications heading [15]. Such resources provide useful discourse supervision, but their labels and document scope differ from ours: a conclusion sentence need not state a gap, and a combined limitations/implications label also covers other content.

More targeted rhetorical schemes come closer to our extraction targets. Sci-Sentence includes Research Gap and Limitation among seven categories intended to support literature-review analysis [16]. Its benchmark samples sentences from introductions, related-work sections and limitations sections. Its Research Gap category overlaps with our open-problem target, whereas its Limitation definition concerns weaknesses of a previous study

rather than exclusively the authors' own work. This difference in attribution matters when deciding which statements count as positives.

Context is another shared concern. Hierarchical sentence classifiers model surrounding discourse [17], and Brack et al. [18] investigate cross-domain sequential classification through multi-task learning. Lan et al. [19] extend sequential classification to LLMs and multi-label outputs, while Zhang and Zhang [20] use context-enhanced representations to identify problem and method sentences. These studies motivate looking at neighbouring text when a sentence is ambiguous. Our funnel does this in its last stage: the LLM filter sees each candidate together with its neighbouring sentences before it decides.

2.2. Recognition of limitations and future work

Work on self-acknowledged limitations directly addresses one of our three targets. Kilicoglu et al. [1] formulate limitation recognition in clinical research as sentence-level binary classification and investigate both rules and learned classifiers. Their rules rely on document structure, such as section headings and the passages in which limitations are discussed, which is a direct precedent for our Stage A. Lan et al. [2] subsequently annotate limitations in 200 RCT publications and separate detecting a limitation from assigning its type. Their fine-tuned PubMedBERT detector achieves a reported F_1 of 0.821 under their evaluation protocol. We use their corpus, SAL, as our human-labelled limitation benchmark, and the PubMedBERT model family as a supervised comparison. Their published score cannot be compared directly with ours, because we score every sentence of the full paper, and the pool of sentences and its class balance affect precision and F_1.

Future-work recognition has a parallel history. Hu and Wan [21] use regular expressions to extract future-work statements, classify them into four categories and expose the results through a search prototype. Zhu et al. [3] use BERT-based recognition to identify future-research sentences in JASIST articles and cluster the extracted statements. The ACL FWS-RC corpus provides annotations and a taxonomy for future-work sentences in NLP papers [22]. Using this resource, Zhang et al. [4] recognise future-work sentences with feature-based classifiers, the strongest being Bernoulli naive Bayes (macro-F_1 of 0.9073), and classify their directions with neural models. Only their recognition task matches ours. Their score is also a macro-F_1, whereas we report F_1 for the positive class, and their data split differs from ours. These published scores therefore provide context, not a ranking against our results.

Extracted statements also support analysis beyond detection. Suray et al. [23] examine how subsequent usable-security research engages with previously stated future work. More recently, Thaepprasit et al. [24] analyse self-reported limitations in ACL and EMNLP papers through iterative human–AI qualitative coding. These two studies show the value of author-written statements for studying a field. At the downstream ideation stage, Si et al. [25] find in a controlled study with more than 100 NLP researchers that LLM-generated ideas were judged more novel but slightly less feasible than expert ideas. Judging ideas is thus a hard problem of its own, which is why we evaluate extraction separately from the ideas that may later be produced from it. Our work addresses only this extraction step; it does not establish whether a stated gap remains unresolved or whether a proposed direction is useful.

2.3. LLM extraction, inferred gaps and generated critique

LLMs are increasingly used both to extract statements and to produce new assessments of research. BAGELS constructs limitation data from ACL, NeurIPS and PeerJ papers, supplements author-written material with external reviews, and studies retrieval-augmented limitation generation [26]. FutureGen uses LLM-extracted author future-work statements,

augmented with reviewer suggestions, as the reference for generating and evaluating new research directions [27]. Cheng et al. [28] directly investigate prompted LLM extraction of future-work sentences from the full text of NLP papers and connect the extracted material to keyword-based research-direction analysis. Together, these studies establish that LLM extraction of author-stated gaps already has close precedents.

Knowledge-gap identification also includes an inference task. GAPMAP explicitly distinguishes stated biomedical knowledge gaps from missing knowledge inferred from context, evaluating LLMs in paragraph-level and full-paper settings [29]. This distinction is central to our scope: we select an existing sentence that articulates a gap and preserve its wording. We do not ask the model to construct an implicit gap that has no corresponding author-written statement.

In limitation generation and scientific critique, the model writes the output rather than selecting existing sentences. LimGen studies suggestive limitation generation using 4068 ACL papers and their associated author-written limitations [30]. Despite its similar name, LimitGen is a separate benchmark for evaluating LLM-generated limitations and feedback on AI research papers [5]. Tree-of-Concerns focuses specifically on unstated limitations through structured multi-agent critique [6]. These tasks can produce useful criticisms beyond those acknowledged by the authors. Our LLM instead returns a keep-or-drop decision for each candidate sentence; the resulting extraction remains attributable to the original text.

2.4. Position of this work

Our Stage B builds on established methodology. Yin et al. [31] formulate zero-shot text classification as textual entailment, allowing a label to be expressed as a natural-language hypothesis. We apply this formulation to gap types using an existing NLI model. Here, zero-shot means that the classifier is not fitted on gap-detection training examples; the model has already been trained on NLI data, and our decision threshold is selected with labelled development data.

Ma et al. [32] provide the closest general information-extraction analogue: a small fine-tuned model filters examples and an LLM reranks the difficult subset. Their system routes by estimated difficulty, whereas our Stage B selects possible gap statements and Stage C makes a binary source-preserving decision. Such staged designs belong to the broader family of LLM cascades, which trade cost against quality; FrugalGPT [33] is a well-known example. Our cascade differs by combining rules, a non-generative classifier and one LLM, instead of routing queries among several LLM services.

Work on long inputs motivates our comparison with whole-paper prompting. Long inputs are a known difficulty for LLMs [7], and RULER finds large performance drops with increasing context length across 17 models [8]. In long-context question answering, order-preserving retrieval can outperform full-context input while using fewer tokens [34]. Closer to our setting, Wu et al. [35] first extract key sentences from a long document and only then prompt an LLM to evaluate a summary, which lowers cost and raises agreement with human judgements. These tasks differ from sentence extraction, but they suggest that selecting candidates first may help an LLM focus on sparse evidence.

Against this background, our contribution is an evaluation of one particular combination: rules, a zero-shot classifier and an LLM filter whose precision–recall trade-off is set by one instruction. We examine how the stages interact, how the filter modes behave across datasets, and how the funnel compares with whole-paper prompting, in accuracy and cost, for LLMs of different sizes. The evaluation covers full papers (SAL and arXiv) as well as conclusion excerpts (FWS). Results for limitations and future work rest on human labels, whereas results for all three gap types together rest on LLM labels and remain provisional.

All of this evidence concerns extraction; assessing the quality of research ideas derived from the extracted statements remains a separate task.

3. Materials and Methods

This section first defines the task and the data. It then describes the three stages of the funnel and how their settings were chosen. Finally, it introduces the systems we compare against and the evaluation protocol.

3.1. Task

Given a paper as a sequence of sentences, the system selects the sentences in which the authors state a research gap. We distinguish three types:

- **Limitation**: a weakness, caveat, bias, constraint or assumption the authors acknowledge about their own work;
- **Future work**: an explicit direction the authors propose for future research;
- **Research gap**: an unsolved problem, missing capability or open challenge the authors point to in the field or in prior work.

The system never rewrites or summarises; each output is an unmodified source sentence. Evaluation is binary at the sentence level: a returned sentence is correct if the reference labels mark it as a gap of a type the benchmark covers, and the predicted type is not scored. SAL labels only limitations and FWS only future work, so on these benchmarks the funnel is asked for that single type.

3.2. Data

We use three datasets (Table 2): two public benchmarks with human labels, each covering one gap type, and a corpus of 230 papers that we labelled for all three types with LLMs. Within each dataset, splits are made by paper.

SAL.

SAL contains 200 RCT reports in which annotators marked the sentences that state a self-acknowledged limitation, split into 120 training, 40 development and 40 test papers [2, 36]. We call the test split SAL-40 and the development split SAL dev. We removed sentences of fewer than three words (none of the removed test sentences is a limitation) and otherwise evaluate on the full paper. To apply section rules in Stage A, we recovered each sentence's section by matching it to the PubMed Central BioC version of its article. The training split is used only to train the supervised classifiers in Section 3.5.

FWS.

The FWS recognition corpus contains sentences from the conclusion sections of ACL, EMNLP and NAACL papers, each labelled as future work or not [4,22,37]. Each "paper" is a conclusion excerpt of about seven sentences, so FWS has no full-paper structure; we use it to test the funnel on a second domain and a second gap type. We built our samples in three steps. We first removed every sentence that also occurs in the training data of an earlier classifier of ours. We then shuffled the papers once. Finally, we took papers in that order until the test sample reached about 1500 sentences (209 papers, 1506 sentences), and continued until the development sample reached about 3000 sentences (the next 418 papers). All FWS text is lowercased before any processing, for every system, because in this corpus letter case reveals the label (Section 4.7.2).

Table 1. Selected related work, ordered by year of the cited version. Target categories describe the relevant part of each study; other rhetorical labels are omitted where applicable. When multiple target categories apply, they are listed in the order Limitations, Future work and Research gaps. "Input unit" is the input each method processes.

Study	Domain	Target categories	Approach	Input unit	Relation to this study
Hu and Wan, 2015 [21]	Computer science	Future work	Rules; multi-class classification	Sentences, full articles	Early extraction, typing and search prototype
Dr. Inventor, 2016 [11]	Computer graphics	Limitations; Future work	Human multi-layer annotation	Sentences, full articles	Full-text precedent with both statement types
Kilicoglu et al., 2018 [1]	Clinical research	Limitations	Rules; supervised classifiers	Sentences	Precedent for section-based localisation
Zhu et al., 2019 [3]	Information science	Future work	BERT classification; clustering	Sentences, full articles	Links recognition to thematic analysis
ACL FWS-RC, 2020 [22]	NLP	Future work	Annotation; taxonomy	Sentences	Source of FWS labels
Zhang et al., 2023 [4]	NLP	Future work	Feature-based recognition; neural typing	Sentences	FWS recognition benchmark
Ma et al., 2023 [32]	General information extraction	IE labels	Small-model filter; LLM reranker	Instances	Closest filter-then-LLM analogue
LimGen, 2024 [30]	NLP	Limitations	LLM generation	Paper	Author limitation sections as references
Lan et al. (SAL), 2024 [2]	RCT reports	Limitations	PubMedBERT detection and typing	Sentences	Source of SAL; supervised comparison
Lan et al. (multi-label), 2024 [19]	Several sciences	Rhetorical roles	LLM prompting and fine-tuning	Abstract sequence	Context-aware LLM labelling
Zhang and Zhang, 2024 [20]	NLP	Problems; Methods	Context-enhanced transformer	Sentence with context	Local context for sparse targets
Brack et al., 2024 [18]	Several sciences	Rhetorical roles	Cross-domain multi-task learning	Sentence sequence	Transfer across domains
Suray et al., 2024 [23]	Usable security	Future work	Statement and citation analysis	Statement	Downstream use of extracted directions
BAGELS, 2025 [26]	NLP; machine learning; biology	Limitations	Extraction; retrieval-augmented generation	Paper; reviews	Extraction and generation benchmark
FutureGen, 2025 [27]	NLP; machine learning	Future work	LLM extraction; retrieval-augmented generation	Paper	Extraction as reference for generation
GAPMAP, 2025 [29]	Biomedicine	Knowledge gaps	LLM detection and inference	Paragraph to paper	Explicit versus inferred gaps
LimitGen, 2025 [5]	AI	Limitations	LLM critique; retrieval	Paper	Evaluates generated limitations
Sci-Sentence, 2026 [16]	Several disciplines	Limitations; Research gaps	Encoder and LLM classifiers	Sentence	Gap labels; different attribution
Tree-of-Concerns, 2026 [6]	Machine learning; NLP; vision	Limitations	Multi-agent critique	Paper	Unstated critique; different target
Cheng et al., 2026 [28]	NLP	Future work	Prompted LLM extraction; keyword analysis	Full text	Direct LLM extraction of future work
Thaepprasit et al., 2026 [24]	NLP	Limitations	Human–AI qualitative coding	Limitation sections	Downstream analysis of reported limitations
This study	RCT reports; NLP; computer science (arXiv)	Limitations; Future work; Research gaps	Rules, zero-shot NLI, LLM filter with three modes	Sentence with context; a paper's candidates	Stage contribution, precision control, comparison with whole-paper prompting

Table 2. Evaluation data. Positives are sentences labelled with a gap type the benchmark covers.

Dataset (split)	Labels	Gap types	Papers	Sentences	Positives	Use
SAL-40 (SAL test)	Human	Limitation	40	7968	205	Main evaluation
SAL dev	Human	Limitation	40	7729	180	Threshold; held-out check
FWS test	Human	Future work	209	1506	220	Evaluation
FWS dev	Human	Future work	418	3005	418	Threshold
Gold test	LLM ensemble	All three	34	11,741	664	Evaluation
Gold validation	LLM ensemble	All three	34	11,201	699	Design choices; thresholds

Gold corpus.

To evaluate all three gap types together, we built the *gold corpus*: 230 papers and 68,719 sentences. It contains 190 arXiv computer-science and machine-learning papers and the SAL-40 papers with their original sentence boundaries. The arXiv papers are a convenience sample collected during earlier development of this project, not a random sample; their identifiers and extracted text are in the repository. They were converted from PDF with PyMuPDF and split into sentences after removing reference lists. We call the corpus "gold" because its labels are our reference for three-type evaluation, but no person has checked them yet, so all results on it are provisional.

The labels were produced in two steps:

1. *Independent labelling.* Two LLMs, Claude Opus 5.5 and Claude Sonnet 5.5, each labelled every sentence of every paper, in one call per paper, following a written guideline with the three definitions above. SAL labels were not shown to them. Both models marked the same 2960 sentences as gaps of the same type.
2. *Adjudication.* The remaining 1737 candidates, marked by only one model or with different types, were judged again by Claude Opus 5.5 with three sentences of context on each side. It confirmed 1502 of them.

The corpus has 4462 gap sentences: 2234 research gaps, 1543 limitations and 685 future-work statements. The two labelling models agreed on whether a sentence is a gap with $\kappa = 0.778$ [38], which measures consistency between the models, not agreement with people. On the 40 SAL papers, the ensemble marked all but nine of SAL's 205 human-labelled limitations as a gap; this shows high recall against SAL, not precision.

Papers were split into 162 training, 34 validation and 34 test papers. The 40 SAL papers are spread over these splits: 28 are in the training split, 6 in the validation split and 6 in the test split. The test split has 664 gaps (220 limitations, 97 future work, 347 research gaps); its 28 arXiv papers hold 527 of them and its 6 SAL papers hold 137. When we report results on "arXiv", we mean these 28 arXiv test papers.

3.3. The funnel

The funnel applies three stages in sequence, each keeping a subset of the sentences it receives (Figure 1).

Stage A: localisation.

Stage A removes text that is unlikely to contain gaps before any model runs. Its rules depend on the dataset, because papers differ in how reliably their sections show where gaps are stated.

- **SAL:** only sentences in Discussion and Conclusion sections are kept.
- **Gold corpus:** no section rule is applied, because in arXiv papers many gaps are stated in the methods and experiments sections (Section 5.4). Stage A instead applies exclusion rules, which we call the *junk filter*. It drops titles, tables, figure captions and

abbreviation lists; competing-interest, author-contribution, funding and supplementary sections where section types are known; bibliography-like strings; and malformed sentences (under 25 characters or six tokens, starting mid-word, containing dot leaders, or consisting mostly of non-letters).

- **FWS:** there is no Stage A. The corpus already consists of conclusion sections, and the junk filter, which rejects sentences that start in lower case, would remove almost all of the lowercased text.

Stage B: zero-shot NLI classifier.

Stage B selects candidates with a classifier that needs no gap-labelled training data. We use an NLI cross-encoder, ModernBERT-large fine-tuned on NLI data [39–41], as a zero-shot classifier [31]: the model reads a sentence (the premise) with a short statement of the label (the hypothesis) and estimates whether the sentence entails it. There is one hypothesis per gap type: "This is a limitation.", "This is future work." and "This is a research gap." For each hypothesis, the entailment and contradiction probabilities are renormalised to sum to one, and the entailment share is the score. On the gold corpus, a sentence becomes a candidate if its highest score over the three hypotheses reaches a threshold τ, and its predicted type is the hypothesis with that score. SAL uses only the limitation hypothesis and FWS only the future-work hypothesis. Inputs are truncated to 256 tokens.

Stage C: LLM filter.

Stage C keeps or drops each candidate with an LLM. Our main model is Qwen3.8-27B (`Qwen/Qwen3.8-27B-FP8`, revision `017b9c7a`), which was not used to create any labels. Candidates are sent in reading order, 20 per request. For each candidate, the model receives its predicted type, the sentence (up to 400 characters) and a context window made of the sentence and its immediate neighbours in the same paper (up to 600 characters). It returns a keep or drop verdict for every candidate, as JSON constrained by a schema.

What the model should keep is set by the system prompt (Appendix A). The prompt defines the three gap types, lists the kinds of sentences to drop (results, method descriptions, contribution claims, background, citations, captions and similar), and contains one paragraph that sets the mode. Only this mode paragraph changes between modes:

- **High recall**: when genuinely uncertain, keep the sentence;
- **Balanced**: favour neither keeping nor dropping;
- **High precision**: keep a sentence only if it explicitly states the gap itself; drop general background, motivation, observed outcomes and passing remarks; when uncertain, drop it.

For SAL and FWS, the definitions are narrowed to the benchmark's single type and the other two types are added to the sentences to drop. The mode paragraphs are identical on all datasets.

The model is self-hosted with vLLM 0.32.0 [42] on one H100 GPU, with temperature 0, seed 0, no reasoning mode and speculative decoding with two draft tokens.

3.4. How settings were chosen

To make clear which data could have influenced each setting, we list where it was chosen.

- **NLI hypotheses.** We compared hypothesis variants (long definitions, short type names, one combined hypothesis, adding neighbouring sentences to the premise, and subtracting competing "not a gap" hypotheses) on one half of the gold validation papers and confirmed the choice on the other half. Short type names worked best.

- **Thresholds.** τ is set on development data only, as the highest threshold at which recall after Stage B is at least 90%, counting positives removed by Stage A or Stage B as missed. With this rule, Stage B keeps most gaps and leaves the removal of false positives to Stage C. The rule gives 0.7286 on SAL dev and 0.1922 on FWS dev.[1] On the gold corpus, $\tau = 0.354$ was set on the half of the validation papers that was also used to choose the hypotheses, and it was then kept fixed. Applying the rule to all 34 validation papers would instead give 0.232, and therefore more candidates.
- **Stage C model, batch size and context.** These and the high-recall instruction were chosen during development on 196 candidates from 27 of the 34 gold test papers. We therefore also report gold results without these candidates.
- **Balanced and high-precision instructions.** These were written after reading false positives of a high-recall run on gold test papers, and checked on the gold validation split before those instructions were run on any test set. All three modes were run on every test set and all are reported, so no mode was selected by its test score. The frozen prompts and run outputs are in the repository history from 5 November 2026; earlier development is not time-stamped.
- **Held-out check.** SAL dev was used only to set τ; no prompt was written or chosen on it. Running the funnel on SAL dev therefore tests it on papers that influenced no prompt.

3.5. Comparison systems

Whole-paper prompting.

The main alternative to the funnel is to give the same LLM the whole paper. The baseline was implemented, and its plan, prompts and code frozen, before any test call. Each call contains one paper as numbered sentences in reading order (one per line, "[id] sentence"), with no Stage A or B, and the model returns the ids of all sentences it judges to be gaps. The system prompt is the funnel's Stage C prompt in the same mode, with only its opening line and the description of the input and output changed. Answers are limited to 4096 tokens; a call that exceeds the limit fails and its gaps count as missed. The arXiv sentence boundaries are those of the gold corpus, so both approaches are scored on exactly the same sentences.

The same frozen plan included two additional baselines: a *chunked baseline* that sends 100 consecutive sentences per call, and an *every-sentence baseline* that sends 20 consecutive sentences per call. The latter judges every sentence in the funnel's batch size, but without Stages A and B. We report both for the 27B model.

The comparison with whole-paper prompting is not perfectly matched, in three ways that matter when reading the results. First, on FWS the frozen plan used a single older prompt from our code base that accepts limitations as well as future work. The FWS baseline therefore has one result for all three modes. This broader prompt can understate the baseline's precision, so the FWS comparison does not isolate the effect of the input strategy. Second, on SAL the funnel uses section labels in Stage A, whereas the whole-paper baseline receives only numbered sentences, which favours the funnel. Third, the arXiv labels were produced by LLMs that read each whole paper in one call, the same input format as the whole-paper baseline, which may favour the baseline. We refer to these three points below as the caveats of the whole-paper comparison.

[1] The stored thresholds are rounded. At the rounded SAL value one development limitation falls below τ, so SAL dev recall before Stage C is 89.4% instead of 90.0%.

Smaller LLMs.

To test how the comparison depends on model size, we repeat the funnel and the whole-paper baseline with Qwen3.5-9B (revision `c2022362`, bfloat16) and Qwen3.5-4B (`Qwen/Qwen3.5-4B`, revision `851bf6e8`, bfloat16). Both are served with vLLM using the same prompts and decoding settings as the 27B model, but without speculative decoding. The 4B model sometimes, and the 9B model once, returned 19 verdicts for a batch of 20 candidates. For such batches we asked again for the missing candidates alone, as the funnel's filter does, and no verdict remained missing. In whole-paper prompting, the 9B model completed every call, whereas the 4B model's answer exceeded the output limit on the longest arXiv paper (Section 4.5).

Published sentence-level methods.

To place the funnel among earlier approaches, we re-ran methods of the kinds used in prior work on exactly our test sentences. None of them uses an LLM.

- **Section and cue rules**, after Kilicoglu et al. [1]. We ported their rules, which combine a gate on the section with cue phrases and rejection patterns. SAL gives no paragraph boundaries, so each section stands in for a paragraph. This is therefore an adaptation of their method, not a reproduction. It targets limitations only, and we run it on SAL.
- **Naive Bayes with selected *n*-grams**, after the best recogniser of Zhang et al. [4], a Bernoulli naive Bayes classifier. Our version uses TF-IDF over word 1- to 4-grams and chi-square selection of 14,000 features. We train it on the raw lowercased text, without their lemmatisation and class balancing, so it is also an adaptation, and we run it on FWS.
- **Two generic supervised classifiers**, run on all three test sets: TF-IDF over word 1- and 2-grams with logistic regression, and Bernoulli naive Bayes on binary 1- to 4-gram features.
- **The NLI model alone**, that is, Stage B's model without Stage A or Stage C. It is run with one broad hypothesis that names all gap types and, on FWS and arXiv, also with the short type hypotheses of Section 3.3.

The supervised methods are trained on training papers only: the 120 SAL training papers, the 134 arXiv training papers of the gold corpus, and 7924 FWS papers outside our test sample. Every method's decision threshold is chosen on separate validation papers and frozen before the test sentences are scored. Differences from the funnel are paired on identical sentences, and all test predictions are in the repository.

Two further attempts are not reported as baselines. We fine-tuned SciBERT and BiomedBERT on the arXiv training papers with a generic three-epoch recipe, but both runs ended by predicting no positive sentence on the test papers, so we regard them as failed training runs. We also could not reproduce the PubMedBERT detector of Lan et al. [2], which was trained with data augmentation, because we did not have its augmentation recipe.

Supervised classifiers on SAL.

To test whether a training-free Stage B is enough, we compare it with classifiers trained on SAL's own labels. We trained eight classifiers on the SAL training split: three fine-tuned encoders, PubMedBERT [43] (`BiomedNLP-BiomedBERT-base-uncased-abstract-fulltext`), SciBERT [44] and DistilBERT [45], each trained for three epochs (learning rate 3×10^{-5}, batch 32, class-weighted loss) with a decision threshold chosen on SAL dev; and five frozen sentence encoders with a logistic-regression classifier (all-MiniLM-L6-v2 and all-mpnet-base-v2 [46], bge-small-en-v1.5 and bge-base-en-v1.5 [47], and e5-base-v2 [48]).

PubMedBERT had the highest F_1 on SAL-40, and we use it as the supervised Stage B. Because this choice was made on the test split, it favours the supervised comparison.

Stage B fine-tuned on LLM labels.

We also fine-tuned the NLI model itself on the gold training split. Each training sentence is paired with each of the three hypotheses, labelled entailment when its gold type matches, neutral when it is a gap of another type and contradiction when it is not a gap. We trained on the 134 arXiv training papers only (33,404 sentences after Stage A, 2534 gaps), leaving out the 28 SAL-40 papers in the gold training split so that SAL-40 stays unseen. Settings were fixed before training: learning rate 10^{-5}, batch 32, two epochs, 6% warm-up, weight decay 0.01, seed 0, one V100 GPU. The epoch with the highest precision on the gold validation split at the 90%-recall rule was kept (the first). Thresholds follow the same rule on each dataset's development data.

Stage C call and answer format.

In the main funnel, each call carries 20 candidates, every candidate comes with its own context, and the model returns a verdict on each one. This format accounts for much of the funnel's LLM cost, so we test three cheaper ones:

- **One call per paper.** All candidates of a paper are sent in one call as numbered sentences, without a context window for each, and the model returns the ids of the sentences to keep. This is the call format of the whole-paper baseline, restricted to the candidates.
- **One call per paper, with the abstract.** The same, with the paper's abstract placed before the candidates as context. The arXiv abstracts were taken from the arXiv API and the SAL abstracts from the papers' abstract sections; FWS has no abstracts.
- **Ids-only answer.** The main funnel's 20-per-call requests, unchanged, except that the model returns the ids of the candidates to keep instead of a verdict on each.

The two one-call-per-paper formats were run with the fine-tuned Stage B.

Other Stage B options on the gold corpus.

We also compare the NLI classifier with two alternatives on the gold corpus: (i) bge-small, a small sentence-embedding model [47], with a logistic-regression classifier trained on the gold training split; and (ii) the same NLI model with one long hypothesis that covers all gap types, used alone without Stage C (threshold 0.75, chosen on validation).

3.6. Evaluation

Metrics.

We report precision, recall and F_1 (their harmonic mean) for the positive class at the sentence level. Recall is computed over all positive sentences in the test papers, including those removed by Stage A, so the losses of every stage count. Sentences from the same paper are not independent, so we resample papers: confidence intervals (CIs) are 95% percentile intervals from 2000 bootstrap resamples of papers [49]. To compare two systems, we score both on the same resampled papers and report the CI of their difference, and call a difference significant when this interval excludes zero. The CIs are not corrected for the number of comparisons, so individual significant differences should be read as descriptive. Comparisons with whole-paper prompting on the gold corpus use its 28 arXiv test papers, on which the baseline was run.

Cost.

We measure cost as the number of LLM input and output tokens reported by the server, and as the summed time of the LLM calls, sent one at a time to one H100 GPU. The 27B whole-paper baseline ran on the second H100 GPU of the same server, in its own vLLM instance with the same model, revision and serving settings; its times are therefore only approximately comparable. The computation of Stages A and B, which use no LLM, is not included. The early 27B runs of the 20-per-call funnel logged neither tokens nor time. Their input is taken from the logged 4B runs, because the prompts and the tokenizer are the same; their output and time are not available.

Run-to-run stability.

To measure how much LLM verdicts vary between identical runs, we repeated the high-recall run on the gold test candidates. In addition, 70 batches of the answer-format experiment were sent twice, which gives a second measure.

Planned human study.

To measure precision against human judgement, we have designed a blind study. Two raters and an independent tie-breaker will label a frozen sample of 135 sentences, drawn from the high-precision output on arXiv and SAL-40 and from decoy sentences. Its results are not part of this version.

3.7. Software and use of generative AI

Experiments used Python with PyTorch, Hugging Face Transformers, sentence-transformers, scikit-learn and vLLM. The repository lock file pins the client environment. The LLM servers ran in a separate environment; the saved run records identify vLLM 0.32.0, but the client lock file does not reproduce the complete server environment. Generative AI is part of the method in two places: Claude Opus 5.5 and Claude Sonnet 5.5 produced the gold-corpus labels, and Qwen models are the Stage C filter and the whole-paper baseline. For the related-work review, LR-AI [50] supported a supplementary search of OpenAlex and arXiv records, deduplication, and title-and-abstract screening; every newly cited source was checked against its primary record. The use of AI assistance in preparing the study and the manuscript is declared at the end of the paper.

4. Results

We report the results in the order of the research questions. For RQ1, we look at what each stage contributes, at the choice of Stage B and at the three filter modes, and we then compare the funnel with published methods. For RQ2, we compare the funnel with whole-paper prompting and test cheaper call formats. For RQ3, we compare LLM labels with human labels. Unless stated otherwise, Stage C uses Qwen3.8-27B in high-precision mode, and results on the gold corpus use its provisional LLM labels.

4.1. Contribution of each stage (RQ1)

Each stage contributes in a different way, and Stage C supplies most of the precision gain (Table 3 and Figure 2).

- **Stage A** removes many sentences only where sections are informative. On SAL-40, keeping only Discussion and Conclusion sections removes 76% of the sentences and loses 2 of the 205 limitations. On the gold corpus, the junk filter removes 18% of the sentences and 3% of the gaps.
- **Stage B** raises precision by a factor of 2.7 to 3.6 and keeps 88–91% of all positives, close to the 90% recall that its threshold targets on development data.

- **Stage C** raises precision by a further 37 percentage points on SAL-40, 28 on FWS and 63 on the gold corpus, and lowers recall by 6, 12 and 34 percentage points. On the gold corpus, most of the lost recall falls on research gaps and limitations (Section 4.3).

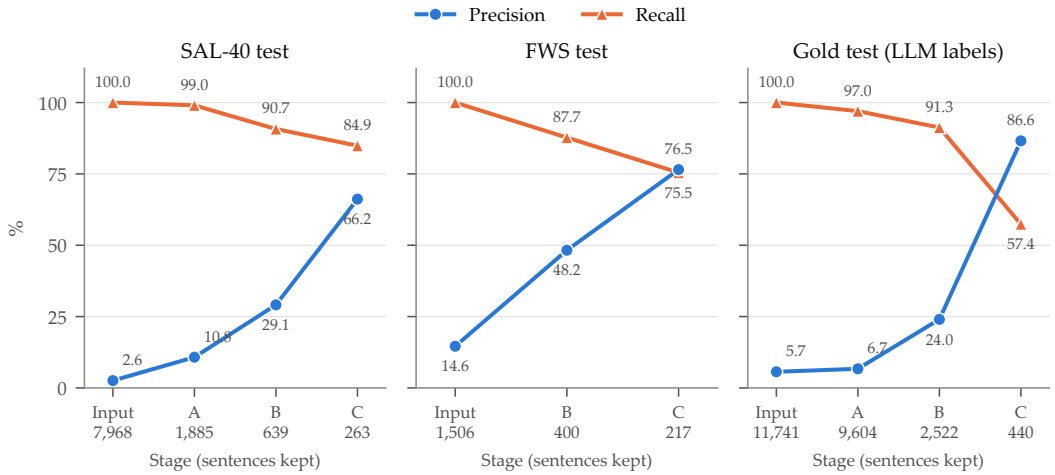


Figure 2. Precision and recall after each stage in high-precision mode. The number below each stage is the number of sentences it keeps. Gold results use LLM labels; FWS has no Stage A.

Table 3. Sentences kept after each stage. High-precision mode; precision and recall in %. Gold results use LLM labels.

Dataset	After stage	Sentences	Positives	Precision	Recall
SAL-40	Input	7968	205	2.6	100
	A. Discussion and Conclusion sections	1885	203	10.8	99.0
	B. NLI, $\tau = 0.7286$	639	186	29.1	90.7
	C. LLM filter	263	174	66.2	84.9
FWS test	Input (no Stage A)	1506	220	14.6	100
	B. NLI, $\tau = 0.1922$	400	193	48.2	87.7
	C. LLM filter	217	166	76.5	75.5
Gold test	Input	11,741	664	5.7	100
	A. Junk filter	9604	644	6.7	97.0
	B. NLI, $\tau = 0.354$	2522	606	24.0	91.3
	C. LLM filter	440	381	86.6	57.4

4.2. Choice of Stage B (RQ1)

Stage B can be filled by different classifiers. We compare the zero-shot NLI model with a classifier trained on human labels, with the same NLI model fine-tuned on our LLM labels, and with two further options on the gold corpus.

Supervised classifier.

We ran the funnel in all three modes with PubMedBERT, trained on SAL's own labels, as Stage B. Before Stage C, PubMedBERT selects 391 candidates with 46.0% precision and 87.8% recall (F_1 0.604). It was the best of the eight SAL-trained classifiers (SciBERT 0.553, DistilBERT 0.523, frozen encoders 0.371–0.457). After Stage C in high-precision mode, it reaches 74.5% precision (95% CI 68.4–80.7), 78.5% recall and F_1 0.765.

Compared with this supervised funnel, the zero-shot funnel is less precise but close in F_1 (Table 4). With PubMedBERT, the funnel is 6.9 to 9.0 percentage points more precise in every mode. In high-precision mode, however, the F_1 difference is small and not significant (-0.021, CI -0.053 to $+0.014$); in the other two modes PubMedBERT's advantage is larger

(0.035 and 0.059). The training-free classifier is therefore competitive in F_1 in high-precision mode, but not in precision. Note that the NLI model's weights were not fitted on any SAL label; only its threshold was set on SAL dev.

Table 4. SAL-40, NLI versus PubMedBERT as Stage B. Left: the funnel with NLI minus the funnel with PubMedBERT in the same mode, paired over resampled papers; precision differences in percentage points, 95% CI in brackets. Right: precision of the NLI funnel's output against the LLM labels instead of SAL's human labels (Section 4.7.1), in %.

	NLI minus PubMedBERT		Precision against LLM labels	
Mode	**Precision**	F_1	**Limitation**	**Any gap type**
High precision	−8.4 (−11.8, −4.5)	−0.021 (−0.053, +0.014)	93.2	95.1
Balanced	−6.9 (−11.5, −2.3)	−0.035 (−0.068, 0.000)	88.8	91.2
High recall	−9.0 (−12.7, −5.1)	−0.059 (−0.089, −0.027)	81.8	85.2

Fine-tuning on LLM labels.

Fine-tuning the NLI model on the gold training labels makes Stage B more precise against those labels, but not against human labels. On the gold test split, under the same 90%-recall rule, it cuts the candidates from 2522 to 1998 and raises their precision from 24.0% to 30.4% at 91.4% recall. On SAL-40 and FWS, in contrast, it is slightly worse than the zero-shot model: 702 instead of 639 candidates at 26.6% instead of 29.1% precision on SAL-40, and 45.7% instead of 48.2% precision on FWS. The same pattern holds after Stage C (Table 10). The fine-tuned funnel improves F_1 on the 34 gold test papers by 0.025 to 0.027 (significant in the high-recall and balanced modes) and on the 28 arXiv papers alone by 0.028 to 0.036 (significant only in high-recall mode), but not on SAL-40 or FWS (−0.019 to +0.002, none significant). One explanation is that the model learned how the LLM labellers mark gaps; differences in domain and input form between the training papers and the benchmarks may also contribute.

Other options on the gold corpus.

On the half of the gold validation papers that was not used to choose the hypotheses, the three short hypotheses reached 39.2% precision at about 79% recall, whereas the bge-small classifier trained on the gold training split reached 28.2% at the same recall. On the test split, the funnel in high-precision mode has higher F_1 than the NLI model used alone with one long hypothesis (0.690 against 0.614; difference 0.076, CI 0.020 to 0.132). It does not differ significantly from a funnel that uses the bge-small classifier as Stage B with the high-recall filter (F_1 0.665; difference +0.025, CI −0.037 to +0.087).

4.3. Precision control with three filter modes (RQ1)

Changing the mode paragraph of the prompt moves the filter along the precision–recall trade-off in the intended direction on every evaluation set (Table 5 and Figure 3). From high recall to balanced to high precision, precision rises and recall falls. In high-precision mode, precision is 66.2% on SAL-40, 76.5% on FWS and 86.6% on the gold test split (83.4% on its 28 arXiv papers and 97.1% on its six biomedical papers).

Which mode has the highest F_1 depends on the dataset. On SAL-40, high precision has the highest F_1 (0.744, against 0.716 for balanced and 0.677 for high recall). On FWS, precision rises from 69.1% to 76.5% while recall falls from 83.2% to 75.5%, so F_1 stays within 0.007. On the gold test split, balanced has the highest F_1: it is 0.055 above high recall (paired 95% CI 0.025 to 0.080) and 0.045 above high precision (0.006 to 0.095).

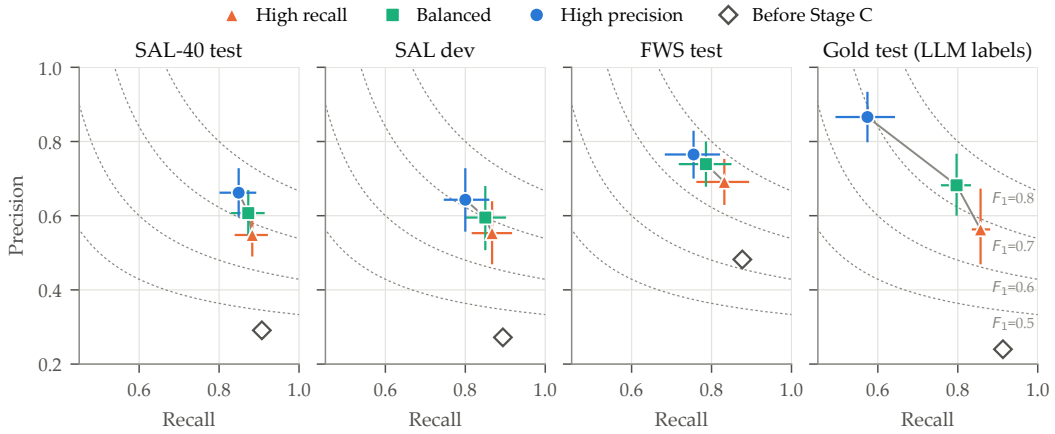


Figure 3. Precision against recall for the three filter modes and for the Stage B output before filtering, with 95% confidence intervals from resampling papers. Dotted curves connect points of equal F_1. Gold results use LLM labels.

Table 5. Results of the three filter modes. Precision and recall in %, with 95% confidence intervals from resampling papers. Gold results use LLM labels. Kept: number of sentences returned.

Evaluation set	Mode	Kept	Precision	Recall	F_1
SAL-40 test (human labels)	Before Stage C	639	29.1 (26.2–32.1)	90.7 (86.7–94.5)	0.441 (0.404–0.475)
	High precision	263	66.2 (59.7–72.8)	84.9 (80.1–89.3)	0.744 (0.695–0.789)
	Balanced	295	60.7 (54.6–66.9)	87.3 (83.0–91.4)	0.716 (0.670–0.760)
	High recall	330	54.8 (49.0–61.3)	88.3 (83.9–92.2)	0.677 (0.627–0.725)
SAL dev (human labels)	Before Stage C	591	27.2 (23.3–31.5)	89.4 (85.1–93.5)	0.418 (0.369–0.468)
	High precision	224	64.3 (55.7–72.8)	80.0 (74.6–86.0)	0.713 (0.647–0.774)
	Balanced	257	59.5 (50.7–68.0)	85.0 (79.9–90.2)	0.700 (0.628–0.764)
	High recall	282	55.3 (46.9–63.9)	86.7 (81.6–91.7)	0.675 (0.602–0.742)
FWS test (human labels)	Before Stage C	400	48.2 (42.2–54.5)	87.7 (81.1–93.4)	0.623 (0.565–0.677)
	High precision	217	76.5 (70.0–82.9)	75.5 (68.3–82.1)	0.760 (0.705–0.807)
	Balanced	234	73.9 (67.8–80.0)	78.6 (71.8–85.0)	0.762 (0.709–0.808)
	High recall	265	69.1 (62.9–75.3)	83.2 (76.2–89.4)	0.755 (0.702–0.804)
Gold test, all (LLM labels)	High precision	440	86.6 (79.8–93.4)	57.4 (49.4–64.3)	0.690 (0.619–0.748)
	Balanced	776	68.2 (60.0–76.7)	79.7 (75.8–83.3)	0.735 (0.678–0.789)
	High recall	1,011	56.3 (46.9–67.3)	85.7 (83.5–88.1)	0.679 (0.604–0.756)
Gold test, arXiv (LLM labels)	High precision	337	83.4 (76.0–91.3)	53.3 (44.4–61.1)	0.650 (0.573–0.713)
	Balanced	643	64.2 (56.0–73.6)	78.4 (73.8–82.6)	0.706 (0.647–0.765)
	High recall	862	51.7 (42.8–62.7)	84.6 (82.1–87.2)	0.642 (0.567–0.719)
Gold test, biomedical (LLM labels)	High precision	103	97.1 (94.5–99.0)	73.0 (63.7–81.0)	0.833 (0.761–0.885)
	Balanced	133	87.2 (82.4–92.2)	84.7 (80.5–87.7)	0.859 (0.833–0.889)
	High recall	149	82.6 (80.2–86.1)	89.8 (85.7–94.4)	0.860 (0.839–0.894)

By gap type.

High precision loses recall mainly on research gaps and limitations (Table 6). Future-work recall barely changes across modes (82.5–86.6%), whereas research-gap recall falls from 89.3% in high-recall mode to 51.3% in high-precision mode, and limitation recall from 79.5% to 55.9%. Research gaps are about half of the gold labels, so they weigh heavily on the overall scores. To see the effect, we removed the sentences labelled as research gaps before scoring, so that they count neither as correct nor as errors. High precision then has the highest F_1 of the three modes (0.701, against 0.595 for balanced and 0.509 for high recall).

Table 6. Gold test split by gap type (LLM labels). Recall for each type, and scores after the sentences labelled as research gaps are removed before scoring; in % except F_1.

	Recall by type			Without research gaps		
Mode	**Limitation (220)**	**Future work (97)**	**Research gap (347)**	**Precision**	**Recall**	F_1
High precision	55.9	82.5	51.3	77.5	64.0	0.701
Balanced	71.4	84.5	83.6	49.2	75.4	0.595
High recall	79.5	86.6	89.3	36.9	81.7	0.509

Held-out checks.

The ordering of the modes also holds on data that played no part in writing the prompts. On SAL dev, whose papers influenced no prompt, high-precision mode reaches 64.3% precision, 80.0% recall and F_1 0.713, close to SAL-40. On the gold validation split, where the modes were checked before the test runs, high-precision mode kept 60.3% of the labelled gaps among the Stage B candidates and rejected 97.7% of the other candidates, against 94.1% and 82.9% in high-recall mode. High precision was thus the stricter mode there as well.

Robustness on the gold test split.

Part of the gold test split was used during development (Section 3.4), so we check how much the results depend on it. Removing the 196 candidates used during development lowers F_1 slightly in every mode: from 0.690 to 0.675 in high-precision mode, from 0.735 to 0.723 in balanced mode and from 0.679 to 0.664 in high-recall mode. A stricter check removes all 27 papers these candidates came from, which leaves 7 test papers with 111 gaps. On these papers, F_1 is 0.575 in high-precision mode (93.9% precision at 41.4% recall), 0.781 in balanced mode and 0.786 in high-recall mode. On untouched papers, high-precision mode thus keeps its precision but loses more recall than on the full test split, although seven papers are too few for firm conclusions. Finally, LLM verdicts vary little between identical runs. Repeating the high-recall run with the same inputs and seed changed 32 of the 2522 verdicts (1.3%), and F_1 was 0.678 in one run and 0.679 in the other.

4.4. Comparison with published methods (RQ1)

The funnel combines three components. A natural question is whether simpler methods from earlier work would do as well on the same sentences. Table 7 compares the funnel with the published sentence-level methods of Section 3.5, all scored on exactly the same test sentences.

The answer differs by dataset.

- **SAL-40.** The funnel is far ahead of every baseline. The best one, the section and cue rules, reaches F_1 0.472 against 0.744 for the funnel, and all differences are significant. The naive Bayes classifier marks only two sentences, both wrongly.

Table 7. Published sentence-level methods against the funnel on identical test sentences. Precision and recall in %. Last column: F_1 of the funnel minus F_1 of the method, paired over resampled papers, with its 95% CI; bold: the CI excludes zero. The rules and the method after Zhang et al. are adaptations (Section 3.5). Kept: number of sentences returned. arXiv results use LLM labels.

Test set	Method	Kept	Precision	Recall	F_1	Funnel − method, F_1
SAL-40 (human labels)	Section and cue rules, after Kilicoglu et al. [1]	210	46.7	47.8	0.472	+**0.271** (+0.171, +0.387)
	TF-IDF with logistic regression	165	47.3	38.0	0.422	+**0.322** (+0.252, +0.392)
	Bernoulli naive Bayes on binary *n*-grams	2	0.0	0.0	0.000	+**0.744** (+0.695, +0.789)
	NLI model alone, one broad hypothesis	363	33.1	58.5	0.422	+**0.321** (+0.274, +0.373)
	Funnel, high precision	263	66.2	84.9	**0.744**	–
FWS (human labels)	Naive Bayes with selected *n*-grams, after Zhang et al. [4]	195	76.9	68.2	0.723	+0.037 (−0.003, +0.079)
	TF-IDF with logistic regression	192	80.2	70.0	0.748	+0.012 (−0.031, +0.054)
	Bernoulli naive Bayes on binary *n*-grams	172	80.8	63.2	0.709	+**0.051** (+0.011, +0.092)
	NLI model alone, one broad hypothesis	282	38.6	49.5	0.434	+**0.325** (+0.264, +0.383)
	NLI model alone, short type hypotheses	218	71.1	70.5	0.708	+**0.052** (+0.012, +0.094)
	Funnel, high precision	217	76.5	75.5	**0.760**	–
arXiv (28 papers, LLM labels)	TF-IDF with logistic regression	667	44.2	56.0	0.494	+**0.156** (+0.080, +0.224)
	Bernoulli naive Bayes on binary *n*-grams	33	48.5	3.0	0.057	+**0.593** (+0.509, +0.661)
	NLI model alone, one broad hypothesis	750	49.9	71.0	0.586	+0.065 (−0.013, +0.140)
	NLI model alone, short type hypotheses	943	41.2	73.8	0.529	+**0.121** (+0.037, +0.196)
	Funnel, high precision	337	83.4	53.3	**0.650**	–

- **arXiv.** The funnel is ahead of all four baselines. The difference is significant for three of them, but not for the NLI model alone with the broad hypothesis (F_1 0.586; difference +0.065, CI −0.013 to +0.140). Three of the four baselines keep two to three times as many sentences as the funnel, at about half its precision.
- **FWS.** Here trained classifiers do well. TF-IDF with logistic regression reaches F_1 0.748 and the naive Bayes adaptation after Zhang et al. reaches 0.723. The funnel's advantage over both is small and not significant (+0.012 and +0.037). FWS provides 7924 training papers from the same venues as the test sample, which favours trained classifiers, whereas the funnel uses FWS labels only to set its threshold.

In short, the funnel is ahead of these methods on full papers, where gap sentences are rare, and clearly so on SAL-40. On FWS, where every text is a conclusion section and training data are plentiful, it is about as good as a trained classifier.

4.5. Funnel versus whole-paper prompting (RQ2)

The natural alternative to the funnel is to give the LLM the whole paper. Table 8 and Figure 4 compare the two approaches with the same model and mode. Two points should be kept in mind when reading them. First, these comparisons were designed after the main results were known, although the baseline was frozen before any of its test calls. Second, the three caveats of Section 3.5 apply: the section labels on SAL favour the funnel, the broader FWS baseline prompt favours the funnel, and the arXiv labels may favour the baseline. We therefore read the differences as descriptions of these particular setups, not as isolated effects of the input strategy.

With the 27B LLM.

With Qwen3.8-27B, the funnel scores higher than whole-paper prompting on both human-labelled benchmarks and lower on arXiv. On SAL-40, the funnel's F_1 is higher by 0.014 to 0.043 across modes, but no difference is significant. In high-precision mode the funnel reaches 66.2% precision and 84.9% recall, and whole-paper prompting reaches 64.1% and 77.6%. On FWS, the difference is 0.060 to 0.068 and significant in every mode,

Table 8. Funnel versus whole-paper prompting. F_1 of each with the same LLM and mode, and their paired difference (funnel minus whole paper) with its 95% CI below it; bold: the CI excludes zero. The intervals are not corrected for multiple comparisons. On FWS the whole-paper baseline has one broader prompt for all modes (Section 3.5). arXiv results use LLM labels.

		Qwen3.8-27B			Qwen3.5-9B			Qwen3.5-4B		
Set	**Mode**	Whole	Funnel	Funnel − whole	Whole	Funnel	Funnel − whole	Whole	Funnel	Funnel − whole
arXiv (28 papers, LLM labels)	High precision	0.679	0.650	−0.029 [−0.10, +0.04]	0.313	0.599	+0.286 [−0.03, +0.42]	0.296	0.544	**+0.248** [+0.07, +0.35]
	Balanced	0.749	0.706	−0.043 [−0.09, +0.02]	0.236	0.564	**+0.328** [+0.04, +0.41]	0.314	0.524	**+0.210** [+0.04, +0.32]
	High recall	0.741	0.642	**−0.099** [−0.16, −0.03]	0.403	0.556	+0.153 [−0.06, +0.25]	0.275	0.480	**+0.205** [+0.01, +0.27]
SAL-40 (human labels)	High precision	0.702	0.744	+0.042 [−0.01, +0.10]	0.684	0.690	+0.007 [−0.07, +0.09]	0.661	0.650	−0.012 [−0.08, +0.06]
	Balanced	0.673	0.716	+0.043 [−0.01, +0.09]	0.680	0.684	+0.004 [−0.07, +0.09]	0.618	0.611	−0.007 [−0.09, +0.07]
	High recall	0.663	0.677	+0.014 [−0.04, +0.06]	0.665	0.607	−0.059 [−0.15, +0.03]	0.599	0.576	−0.023 [−0.10, +0.04]
FWS (human labels)	High precision	0.694	0.760	**+0.065** [+0.02, +0.11]	0.654	0.731	**+0.077** [+0.03, +0.12]	0.719	0.727	+0.009 [−0.04, +0.07]
	Balanced	″	0.762	**+0.068** [+0.02, +0.12]	″	0.725	**+0.071** [+0.03, +0.11]	″	0.711	−0.008 [−0.06, +0.05]
	High recall	″	0.755	**+0.060** [+0.02, +0.11]	″	0.728	**+0.074** [+0.04, +0.11]	″	0.731	+0.013 [−0.04, +0.06]

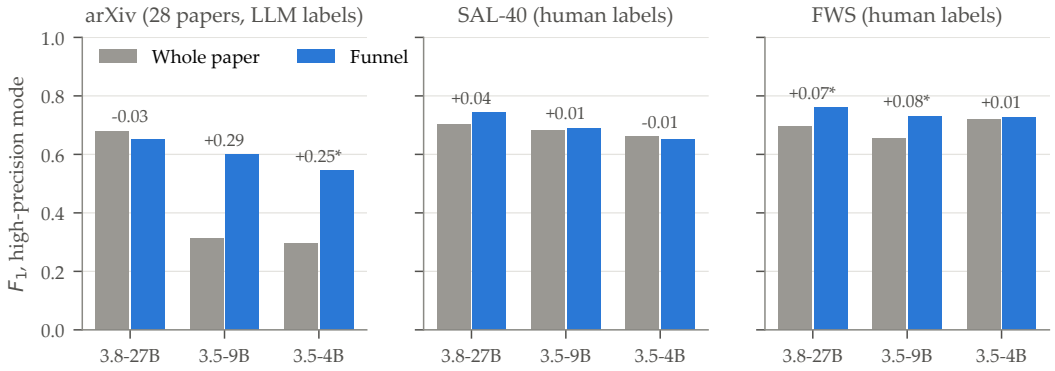


Figure 4. F_1 in high-precision mode for whole-paper prompting (grey) and the funnel (blue) with each LLM. Numbers give the difference (funnel minus whole paper); an asterisk marks a paired 95% CI that excludes zero. arXiv results use LLM labels.

subject to the caveat about the broader baseline prompt. On the arXiv papers the order reverses: whole-paper prompting is better in every mode, significantly so in high-recall mode (-0.099).

The two additional baselines follow the same pattern as whole-paper prompting. The chunked baseline reaches F_1 0.70–0.73 on arXiv and 0.60–0.68 on SAL-40, and the every-sentence baseline reaches 0.70–0.72 and 0.55–0.64. Both are ahead of the funnel on arXiv and behind it on SAL-40, and both use more input tokens than the whole paper.

With the 9B LLM.

With Qwen3.5-9B, whole-paper prompting breaks down on long arXiv papers. The model completes every call but selects far too much on several long papers, keeping 806 of 824 sentences in one mode and 499 of 692 in another. Across modes, its whole-paper F_1 on arXiv is 0.236–0.403, against 0.556–0.599 for the funnel. The difference is large (0.153–0.328), but it is significant only in balanced mode, because the failures are concentrated in a few papers. On SAL-40, the difference ranges from -0.059 to $+0.007$ and is never significant. On FWS, it is 0.071–0.077 and significant in every mode, again subject to the caveat about the baseline prompt.

With the 4B LLM.

With Qwen3.5-4B, the funnel helps clearly on long papers. Whole-paper prompting performs poorly on arXiv: in high-precision mode it reaches 20.7% precision and 52.2% recall (F_1 0.296). On long papers the model returns most of the paper, for example 490 of 692 sentences. On the longest paper (1526 sentences, 83 gaps), its answer exceeded the output limit in every mode. This failure is not the main cause, however: without that paper, F_1 is still 0.29–0.33. The funnel, which never sends the model more than 20 candidates at a time, reaches F_1 0.544 with the same model. Its advantage is 0.21 to 0.25 and significant in every mode. On SAL-40 and FWS, whose inputs are shorter, the two approaches do not differ significantly.

Paper length.

The same pattern appears within one dataset when the arXiv papers are split at their median length (239 sentences). We ran this analysis for the 27B and 4B models in high-precision mode. On the 14 shorter papers, whole-paper prompting is ahead with both models: F_1 is 0.707 for whole-paper prompting against 0.624 for the funnel with the 27B model, and 0.658 against 0.595 with the 4B model. On the 14 longer papers, the two approaches are level with the 27B model (0.659 against 0.668) and far apart with the 4B model (0.203 against 0.516). Within these models and papers, the funnel's benefit therefore grows with input length and is larger for the 4B model.

Cost.

The funnel reads less input than whole-paper prompting, but how much less depends on the dataset (Table 9). On SAL-40 and FWS, it uses about a third of the input tokens. On arXiv the saving is small: whole-paper prompting uses only 1.4 times the funnel's input, because each of the 2137 candidates is sent with its own context. The funnel's output is larger, because it returns a verdict for every candidate instead of a list of kept sentences. With the 9B and 4B models, this makes the 20-per-call funnel slower than whole-paper prompting on arXiv (287 against 92 seconds and 198 against 160 seconds). The chunked and every-sentence baselines are more expensive than the whole paper: with the 27B model they used 0.58–0.76 million input tokens on arXiv and 0.36–0.48 million on SAL-40. The last row for each dataset in Table 9 shows a cheaper call format, which Section 4.6 introduces.

Table 9. Measured LLM cost in high-precision mode: input tokens (the same prompts for every model), output tokens and summed call time in seconds, one call at a time on one H100 GPU. arXiv: the 28 arXiv test papers; for the 20-per-call funnel, whose batches mix arXiv and SAL papers, each batch is counted by its share of arXiv candidates. The one-call-per-paper funnel uses the fine-tuned Stage B (Section 4.6). The early 27B runs of the 20-per-call funnel logged neither output tokens nor time (n/a). The 4B model's logged whole-paper input on arXiv is lower (482k), because its failed call on the longest paper was not logged.

			Qwen3.8-27B		**Qwen3.5-9B**		**Qwen3.5-4B**	
Set	**Setup**	**Input**	Output	Seconds	Output	Seconds	Output	Seconds
arXiv	Whole paper	554k	1.6k	104	5.3k	92	6.0k	160
	Funnel, 20 per call	403k	n/a	n/a	23k	287	23k	198
	Funnel, one call per paper	75k	1.4k	31	2.1k	32	2.1k	24
SAL-40	Whole paper	348k	1.5k	65	1.1k	37	1.5k	30
	Funnel, 20 per call	106k	n/a	n/a	7.4k	89	6.4k	54
	Funnel, one call per paper	42k	1.1k	27	0.9k	21	0.9k	20
FWS	Whole paper	183k	2.7k	89	1.9k	80	2.1k	72
	Funnel, 20 per call	62k	n/a	n/a	4.0k	49	4.0k	34
	Funnel, one call per paper	67k	1.3k	59	1.2k	51	1.1k	47

4.6. Stage C call and answer format (RQ2)

The cost figures above show that the main funnel spends many tokens on context windows and on one verdict per candidate. We therefore tested the cheaper formats of Section 3.5. Table 10 gives their F_1 next to whole-paper prompting.

Table 10. Funnel variants. F_1 of whole-paper prompting and of three funnels, by LLM; each cell gives high precision / balanced / high recall. On FWS the whole-paper baseline has one prompt for all modes. arXiv results use LLM labels.

Set	**Setup**	**Qwen3.8-27B**	**Qwen3.5-9B**	**Qwen3.5-4B**
arXiv	Whole paper	0.679 / 0.749 / 0.741	0.313 / 0.236 / 0.403	0.296 / 0.314 / 0.275
	Funnel (zero-shot NLI, 20 per call)	0.650 / 0.706 / 0.642	0.599 / 0.564 / 0.556	0.544 / 0.524 / 0.480
	Fine-tuned NLI, 20 per call	0.687 / 0.734 / 0.674	0.615 / 0.603 / 0.591	0.591 / 0.595 / 0.514
	Fine-tuned NLI, one call per paper	0.642 / 0.687 / 0.691	0.591 / 0.630 / 0.607	0.484 / 0.460 / 0.530
SAL-40	Whole paper	0.702 / 0.673 / 0.663	0.684 / 0.680 / 0.665	0.661 / 0.618 / 0.599
	Funnel (zero-shot NLI, 20 per call)	0.744 / 0.716 / 0.677	0.690 / 0.684 / 0.607	0.650 / 0.611 / 0.576
	Fine-tuned NLI, 20 per call	0.725 / 0.713 / 0.674	0.669 / 0.637 / 0.601	0.660 / 0.611 / 0.565
	Fine-tuned NLI, one call per paper	0.720 / 0.705 / 0.673	0.712 / 0.705 / 0.700	0.711 / 0.708 / 0.697
FWS	Whole paper	0.694 / 0.694 / 0.694	0.654 / 0.654 / 0.654	0.719 / 0.719 / 0.719
	Funnel (zero-shot NLI, 20 per call)	0.760 / 0.762 / 0.755	0.731 / 0.725 / 0.728	0.727 / 0.711 / 0.731
	Fine-tuned NLI, 20 per call	0.759 / 0.764 / 0.736	0.706 / 0.728 / 0.724	0.713 / 0.693 / 0.703
	Fine-tuned NLI, one call per paper	0.749 / 0.736 / 0.723	0.743 / 0.740 / 0.741	0.725 / 0.721 / 0.741

One call per paper.

Sending all of a paper's candidates in one call makes the funnel much cheaper. Its input falls to one-seventh of whole-paper prompting on arXiv and to about one-eighth on SAL-40, with no more output, and whole-paper prompting takes 1.5 to 7 times as much LLM time (Table 9). How much accuracy this format costs depends on the model (Table 10).

- **27B.** The one-call format is slightly less accurate than the fine-tuned funnel with 20 candidates per call. The difference is -0.001 to -0.008 on SAL-40, and 0.01 to 0.05 on FWS and in two of the three arXiv modes.

- **9B.** The one-call format reaches F_1 0.591–0.630 on arXiv, 0.700–0.712 on SAL-40 and 0.740–0.743 on FWS. Compared with whole-paper prompting, the difference is significant in arXiv balanced mode and in every FWS mode.
- **4B.** The one-call format is the best or tied-best setup on the human-labelled benchmarks. On SAL-40 it is ahead of whole-paper prompting by 0.050 to 0.098 (significant in the balanced and high-recall modes), with higher precision (67.1% against 56.8% in high-precision mode).

Placing the paper's abstract before the candidates did not help. With the 27B model, it changed F_1 by between -0.024 and 0.000.

Answer format.

Asking for the list of candidates to keep, instead of a verdict on each, cuts output tokens four- to eightfold when everything else is unchanged. It also changes the result, however: 7.6–9.3% of the verdicts differ, far more than the 0.4% by which 70 identical batches differed when they were run twice. The model keeps more candidates in this format. Precision in high-precision mode falls from 86.6% to 79.2% on the gold test split, at a similar F_1 (differences of -0.023 to $+0.006$, none significant).

4.7. LLM labels versus human labels (RQ3)

We answer RQ3 on the datasets that have human labels. On the 40 SAL test papers, the LLM labels include all but nine of SAL's 205 human-labelled limitations (Section 3), so they rarely miss what the annotators marked. They also mark many sentences that SAL leaves unlabelled, and this difference dominates the measured precision, as the first check below shows. A classifier fine-tuned on the LLM labels follows the same pattern: it becomes more precise against those labels but slightly less precise against the human labels of SAL and FWS (Section 4.2). The second check concerns the human labels themselves.

4.7.1. Precision depends on the label source

The precision of the same output changes greatly with the reference labels. Of the 263 sentences returned in high-precision mode on SAL-40, SAL's annotators marked 174 as limitations, whereas the LLM ensemble marked 245 as limitations and 250 as a gap of some type. The two label sets disagree mainly on 77 sentences that are gaps according to the LLM labels but not according to SAL; only 12 returned sentences are negative under both. Precision is therefore 66.2% against SAL and 93.2–95.1% against the LLM labels (right side of Table 4). Which value is closer to the truth depends on who is right about these 77 sentences. The planned human study is designed to settle this question.

4.7.2. Letter case reveals the label in FWS

In the FWS recognition corpus as we obtained it (64,896 sentences from 9013 papers), all 9009 positive sentences are entirely lowercase, and almost no negative sentence is. The rule "a sentence is positive if it is all lowercase" therefore reaches 99.8% precision and 100% recall. Our copy is identical to the published file [37] (the same git blob hash), so the artefact is present in the released corpus. Any case-sensitive model, and any LLM that sees the raw text, can exploit this signal. For this reason, all FWS results in this paper use lowercased text.

5. Discussion

We first interpret the answers to the research questions. We then discuss when whole-paper prompting is enough, how reliable the extracted gaps are, why section rules do not

transfer to computer-science papers, and what the results imply for evaluation data. The section ends with the limitations of the study.

5.1. Answers to the research questions

RQ1: each stage has its own role.

Stage A is cheap, but it is useful only where section headings are informative. Stage B sets the upper limit on recall, because Stage C only removes candidates and cannot recover a gap that Stage B missed. Stage C provides most of the gain in precision. Throughout, every reported gap remains a sentence the authors wrote.

RQ1: a training-free Stage B is viable but less precise.

A zero-shot NLI classifier needs no gap-labelled data and works on both biomedical and computer-science text. On SAL, a supervised classifier was 6.9 to 9.0 percentage points more precise in every mode, while the F_1 difference in high-precision mode was small and not significant. This supervised comparison is available only for SAL, and its classifier was selected on the SAL test split. Fine-tuning on LLM labels did not substitute for human labels.

RQ1: one instruction sets the trade-off.

The three modes kept their order on every evaluation set, on the held-out SAL dev papers and on the gold validation split before any test run. Within these datasets and prompts, the mode paragraph therefore provides a practical control. High precision suits idea generation, where every false gap costs a downstream model or a person time, and high recall suits a reviewer who wants to see every candidate. High precision costs recall mostly on research gaps. A possible reason, which we have not tested, is that the high-precision instruction asks the model to drop background and motivation, and authors often state open problems in exactly that role.

RQ1: the combination matters most on full papers.

Compared with published sentence-level methods, the funnel was far ahead on SAL-40 and ahead on arXiv, where gap sentences are rare and spread through long papers. On FWS, which consists of conclusion sections and offers plentiful training data, a TF-IDF classifier with logistic regression came close to the funnel. The funnel is therefore most useful on full papers. On short, pre-selected text with plentiful training data, a trained classifier is a serious alternative.

RQ2: the funnel helps the 9B and 4B models on long papers.

With both Qwen3.5-9B and Qwen3.5-4B, whole-paper prompting performed poorly on long arXiv papers, and the funnel did not. With the 27B model the picture is mixed. The funnel scored higher on the two human-labelled benchmarks, using about one-third of the input tokens, and lower on arXiv papers against LLM labels. The caveats of Section 3.5 limit what the 27B results show, and the SAL differences were not significant. The call format matters for cost as well: sending each paper's candidates in a single call cuts the input on full papers to about one-seventh of whole-paper prompting on arXiv and to one-eighth on SAL-40.

RQ3: LLM labels are not yet a substitute for human labels.

On SAL, the LLM labels include nearly every human-labelled limitation, but they also mark many sentences the annotators left unlabelled. The same output therefore scores 66.2% precision against human labels and 93.2% against LLM labels, and a classifier fine-tuned on LLM labels improves against those labels but not against human ones. Within

these data, precision measured against LLM labels should not be read as precision against human judgement, which is why we call the three-type results provisional. Whether the same holds for future work and research gaps is what the planned human study will test.

5.2. When is whole-paper prompting enough?

Three factors decide the comparison in our data.

- **Input length.** SAL papers have about 200 sentences and FWS excerpts about seven. arXiv papers have a median of 239 sentences and up to 1526, with whole-paper inputs of up to 72,000 tokens. The 9B and 4B models handled the short inputs but selected far too much on several long arXiv papers. In the median-split analysis, which was run for the 27B and 4B models, the funnel helped the 4B model on the longer half and not on the shorter half.
- **Model size and generation.** The 27B model handled every input, whereas the 9B and 4B models both failed on long inputs in a similar way. These two models belong to the same generation, Qwen3.5, so the failure is unlikely to be specific to the 4B checkpoint. The 27B model, however, belongs to the newer Qwen3.8 generation. Size is therefore not the only difference between it and the two smaller models, and our results cannot separate the effect of size from the effect of model design.
- **Reference labels.** The arXiv labels were produced by LLMs that read each whole paper in one call, the same input format as the whole-paper baseline, and may therefore favour it. On the two benchmarks labelled by people, the funnel was ahead with the 27B model.

Taken together, these factors give a simple answer for our data. With the 27B model, whole-paper prompting was a reasonable choice, especially on shorter papers, as long as its higher input cost is acceptable. With the 9B and 4B models, it held up on SAL-40 but failed on several long arXiv papers, and there these models needed the funnel.

The call format also matters. The funnel's main cost on arXiv is that each candidate carries its own context and receives its own verdict. Sending each paper's candidates in a single call removes most of this cost. This one-call-per-paper format is strong with the 9B model on all three datasets and is the most accurate setup on SAL-40 with the 4B model. With the 27B model, however, it loses up to 0.05 F_1 on arXiv, and asking for a list of kept sentences instead of a verdict on each candidate makes the model less selective. Cheaper formats are thus a trade-off, not a free improvement.

5.3. How reliable are the extracted gaps?

How good the extracted gaps are depends on which labels are trusted. On SAL-40, high-precision mode reaches 66.2% precision against SAL's labels and 93.2% against the LLM labels. Either label set may be wrong on the 77 sentences where they disagree. On the one hand, SAL's annotators may have left some limitation statements unlabelled. On the other hand, the LLM labels are inclusive by construction, because adjudication added sentences that only one model had marked. The two values describe the same output under two references, and neither is a bound on its true precision.

Label errors may also affect the gold corpus. In an earlier high-recall run during development, which is not part of the results above, five of the 34 test papers accounted for 59% of the false positives. Each of these papers had only about 18–24 gold gaps despite having 220 to 824 sentences, which may indicate that long papers are under-labelled. If so, some sentences counted as false positives are in fact gaps, and precision against these labels is understated, most of all for systems that keep many sentences. The effect on measured recall is less clear, because unlabelled gaps are missing from both the numerator and the denominator.

The planned human study (Section 3) will measure precision against human judgement on a frozen sample from arXiv and SAL-40.

5.4. Section rules do not transfer to computer-science papers

In RCT reports, limitations cluster in the Discussion and Conclusion sections, so a section rule works. In arXiv papers it did not work in our development experiments. An earlier localiser kept only the sections where gaps are usually stated, such as the discussion and the conclusion. We measured it against the limitation and future-work labels of the arXiv papers, with a fuzzy sentence match. It kept only 55.6% of these gaps when sections were inferred from the extracted text, and 55.2% when sections were taken from the LaTeX sources of 175 papers. Better section information therefore did not help: with LaTeX sections, 82% of the missed gaps were in methods, experiments or results sections. This development analysis used a different target and matching rule from our main evaluation and was not repeated for this version. For arXiv papers we therefore use only the junk filter as Stage A, which keeps 97% of the gold test gaps, and leave the narrowing to Stage B.

5.5. Evaluation data needs checking

The FWS case artefact shows that a published benchmark can contain a trivial signal that a modern model will find. FWS results from case-sensitive models, or from LLMs given the raw text, should be treated with caution unless the text is normalised. More generally, the precision we measure on SAL-40 changed more with the label source than with the choice of Stage B. Stating which labels a number rests on is as important as reporting the number.

5.6. Limitations

- **LLM labels.** All results on the gold corpus rest on labels produced by Claude models, which have not yet been checked by people. The agreement between the two labelling models ($\kappa = 0.778$) measures their consistency, not their accuracy. Claude models also assisted in writing the prompts and this manuscript, so the labels and the prompts may share biases.
- **Development on test papers.** The Stage C model, batch size, use of context and high-recall instruction were chosen on 196 candidates from 27 of the 34 gold test papers, and the balanced and high-precision instructions were written after reading false positives on gold test papers. We report results without these candidates and on the 7 untouched papers, ran and report all modes on every dataset, and checked the modes on the validation split first. Still, the gold test split is not untouched. SAL dev provides the cleanest held-out check, and its results agree with those on SAL-40.
- **Fine-tuned classifier and arXiv scores.** The one-call-per-paper funnel uses a Stage B classifier fine-tuned on the gold training labels, which may favour it when it is scored against the same kind of labels on arXiv. The zero-shot funnel with 20 candidates per call shows a similar gain with the 4B model (F_1 0.544 against 0.296 for whole-paper prompting), so the gain does not depend on fine-tuning.
- **Comparisons designed later.** The comparisons with whole-paper prompting, smaller models, other formats and published methods were designed after the main results were known. The whole-paper baseline and the published-method baselines were frozen before their test runs, and all runs of these comparisons are reported. The comparison with whole-paper prompting is also subject to the three caveats of Section 3.5.
- **Published methods.** The rules after Kilicoglu et al. and the classifier after Zhang et al. are adaptations, not exact reproductions. We could not reproduce the detector of Lan et al., and our fine-tuned SciBERT and BiomedBERT runs on arXiv failed, so these are not among the baselines.

- **Statistics.** Apart from two stability checks (one repeated high-recall run and 70 repeated batches), each configuration was run once, and the many paired comparisons are not corrected for multiplicity. Small differences, especially those whose confidence intervals include zero, should not be treated as established effects.
- **Scope.** SAL covers RCT reports only, FWS covers NLP conclusion sections only, and each labels a single gap type. The six biomedical papers in the gold test split are also SAL papers. The arXiv papers are a convenience sample of 28 test papers. Stage C was tested with one model family, in which the largest model belongs to a newer generation than the two smaller ones.
- **Research gaps.** Half of the gold labels are research gaps, which the earlier stages retain better than the other types. We therefore also report recall by type and scores without research gaps.
- **Cost.** Cost is measured as LLM tokens and call time on our hardware, not in money, and excludes Stages A and B.

6. Conclusions

We presented a three-stage funnel that extracts author-stated research gaps as unmodified source sentences. Rules remove text that is unlikely to contain gaps, a zero-shot NLI classifier selects candidates without gap-labelled training data, and an LLM verifies them in one of three modes set by a single instruction.

Our first question was what each component contributes. The stages have distinct roles: Stage A helps when sections are informative, Stage B sets the upper limit on recall, and Stage C supplies most of the precision gain on full papers. The modes shift precision and recall in the intended direction on every dataset. In high-precision mode, precision is 66.2% at 84.9% recall on SAL-40 and 76.5% at 75.5% recall on FWS. For Stage B, the training-free NLI classifier comes within 0.021 F_1 of a supervised PubMedBERT in high-precision mode on SAL, although it is 8.4 percentage points less precise. Compared with published sentence-level methods, the funnel is far ahead on SAL-40, whereas on FWS a trained TF-IDF classifier comes close to it.

Our second question was how the funnel compares with giving the same LLM the whole paper. The answer depends on the model. With the 9B and 4B LLMs, whole-paper prompting failed on several long arXiv papers, and the funnel was substantially better there, although the 9B difference is significant only in balanced mode. With the 27B LLM, the two approaches are closer: the funnel scores higher on SAL-40 and FWS, and whole-paper prompting scores higher on arXiv, with the caveats discussed in Section 3.5. The funnel is also cheaper. Sending each paper's candidates in a single call reduces the input on full papers to about one-seventh of whole-paper prompting on arXiv and one-eighth on SAL-40, and with the 4B model and the fine-tuned classifier this format raises F_1 on arXiv from 0.30 to 0.48.

Our third question was how far LLM labels can stand in for human labels. In our data they could not. The same SAL output scored 66.2% precision against human labels and 93.2% against LLM labels, and fine-tuning the classifier on LLM labels improved its agreement with those labels but not with the human-labelled benchmarks. For this reason, the results on our gold corpus remain provisional: 86.6% of the kept sentences match its LLM labels at 57.4% recall, but no person has checked these labels yet, and some gold test candidates influenced the design of the funnel.

The next steps are the human validation of the gold corpus, exact reproductions of further published methods, and the use of the extracted gaps for automated research-idea generation.

https://doi.org/10.3390/make1010000

Author Contributions: [To be confirmed by all authors before submission, using CRediT roles.] All authors have read and agreed to this version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: Code, prompts, evaluation scripts, and the gold-corpus labels and paper splits are openly available at https://github.com/Yazangthb/Gap2Idea/tree/preprint-v1 (tag `preprint-v1`) [51]. The gold-corpus labels were produced by language models and have not yet been validated by people. The SAL data [36] and the FWS corpus [37] were created by their authors and are not redistributed. Instead, the repository includes a script that downloads the exact versions we used, pinned to an upstream commit and checked by hash, and a script that rebuilds our SAL and FWS evaluation splits from them, byte for byte. The arXiv papers are available from arXiv; the repository includes the text we extracted from them and their abstracts. All LLM verdicts behind the reported results are included, for the funnel, the whole-paper baseline and every model size, together with the test predictions of the published-method baselines. The FWS split depends on an earlier training file of ours, `data/scibert_prep/train.jsonl` (SHA-256 prefix `f8d5fd58c5b9f1bb`), which is also included.

Acknowledgments: The authors thank Rubetek for supporting this work. During the preparation of this study and manuscript, the authors used Claude Opus 5.5 (Anthropic), through Claude Code, to write and run analysis code and to draft and edit text, and OpenAI Codex to revise text. Claude Haiku 5.5 (Anthropic), within LR-AI [50], screened candidate papers for the related-work review. The authors have reviewed and edited the output and take full responsibility for the content of this publication.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

CI	Confidence interval
FWS	Future work sentence (corpus)
LLM	Large language model
NLI	Natural language inference
NLP	Natural language processing
RCT	Randomized controlled trial
SAL	Self-acknowledged limitations (corpus)

Appendix A. Stage C prompt

The system prompt used on the gold corpus is reproduced below. The three modes differ only in the paragraph marked [MODE], whose three versions are listed after the prompt. For SAL and FWS, the decision rule keeps only item (a) or item (b), respectively; the other two items are added to the list of sentences to drop, and the words "gap" and "research gap" are replaced by "limitation" or "future-work direction". All prompts are in the code repository (`scripts/bench/stage_c_rubric_val.py`, `stage_c_sal40_modes.py`, `nli_sal_fws.py`).

> You are a precision filter for a research-gap extraction pipeline. Judge each candidate sentence: does it genuinely state a research gap?
>
> Each item has: "i" (integer id), "type" (the predicted gap type), "sentence" (the candidate to judge), and "context" (surrounding text). Judge the "sentence", using "context" only to disambiguate.

> Decision — answer YES (keep) if the sentence states ANY of: (a) a limitation / weakness / caveat / bias / constraint / assumption of the authors' OWN work (e.g., small sample, no blinding, single setting, short follow-up, confounding, limited generalizability, unvalidated measure, "we did not / could not . . . ", "a limitation is . . . "); (b) an explicit FUTURE-WORK direction the authors propose ("in future work we will . . . ", "future research should . . . ", "it would be interesting to . . . "); (c) an unsolved problem / missing capability / open research gap in the field or prior work ("it remains unknown . . . ", "no existing method . . . ", "current approaches struggle with . . . ").
>
> Answer NO (drop) if the sentence is instead: a result or finding, a method/experimental-setup description, a positive or contribution claim ("we propose/show/achieve . . . "), background or related-work description, a citation or reference fragment, acknowledgements / funding / author contributions / competing interests, a figure or table caption, a dataset/metric definition, or a generic non-gap statement.
>
> [MODE]
>
> Return JSON of the form {"results":[{"id":<i>, "keep":<bool>}, . . .]} with EXACTLY one entry per input item, where id is the item's "i" and keep=true means YES.

The [MODE] paragraph:

- **High recall:** "Be KEEP-BIASED: when you are genuinely uncertain whether a sentence is a real gap, answer YES. Only answer NO when it clearly is NOT a gap by the rules above."
- **Balanced:** "Judge each sentence on the rules above, without favouring YES or NO."
- **High precision (main mode):** "Be STRICT. Answer YES only when the sentence itself explicitly states the limitation, the proposed direction, or the unsolved problem. Also answer NO for: a general property or difficulty of a method or domain given as background or motivation; a statement of why the authors' approach is needed; an observed experimental outcome; and a passing remark that names no specific gap. When genuinely uncertain, answer NO."

Whole-paper and one-call-per-paper prompts. The whole-paper baseline and the one-call-per-paper funnel use the same prompt with some sentences replaced. The whole-paper prompt also replaces the opening line, which reads "You are reading sentences from scientific papers. Judge each sentence:". The description of the items is replaced by a description of the numbered input ("one per line as [id] sentence"; for the one-call-per-paper funnel, "the candidate sentences of ONE scientific paper . . . Sentences between candidates were removed"), and the output line asks for {"positives":[<id>, . . .]}, the ids of every sentence answered YES, in increasing order. The exact texts are in `scripts/bench/stage_c_per_paper_ft.py` and in the frozen plan of the whole-paper baseline (`docs/experiments/baseline_qwen/prompts/`).

References

1. Kilicoglu, H.; Rosemblat, G.; Malički, M.; ter Riet, G. Automatic Recognition of Self-Acknowledged Limitations in Clinical Research Literature. *Journal of the American Medical Informatics Association* **2018**, *25*, 855–861. https://doi.org/10.1093/jamia/ocy038.
2. Lan, M.; Cheng, M.; Hoang, L.; ter Riet, G.; Kilicoglu, H. Automatic Categorization of Self-Acknowledged Limitations in Randomized Controlled Trial Publications. *Journal of Biomedical Informatics* **2024**, *152*, 104628. https://doi.org/10.1016/j.jbi.2024.104628.
3. Zhu, Z.; Wang, D.; Shen, S. Recognizing Sentences Concerning Future Research from the Full Text of JASIST. *Proceedings of the Association for Information Science and Technology* **2019**, *56*, 858–859. https://doi.org/10.1002/pra2.206.
4. Zhang, C.; Xiang, Y.; Hao, W.; Li, Z.; Qian, Y.; Wang, Y. Automatic Recognition and Classification of Future Work Sentences from Academic Articles in a Specific Domain. *Journal of Informetrics* **2023**, *17*, 101373. https://doi.org/10.1016/j.joi.2022.101373.

5. Xu, Z.; Zhao, Y.; Patwardhan, M.; Vig, L.; Cohan, A. Can LLMs Identify Critical Limitations within Scientific Research? A Systematic Evaluation on AI Research Papers. In Proceedings of the Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Association for Computational Linguistics, 2025, pp. 20652–20706. https://doi.org/10.18653/v1/2025.acl-long.1009.
6. Mishra, S.; Rajeev, N.; Chakraborty, T. Tree-of-Concerns: Hierarchical Multi-Agent Debate for Unstated-Limitation Extraction in Scientific Critique. In Proceedings of the Findings of the Association for Computational Linguistics: EMNLP 2026, Budapest, Hungary, 2026; pp. 2735–2764. https://doi.org/10.18653/2026.findings-emnlp.134.
7. Liu, N.F.; Lin, K.; Hewitt, J.; Paranjape, A.; Bevilacqua, M.; Petroni, F.; Liang, P. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* **2024**, *12*, 157–173. https://doi.org/10.1162/tacl_a_00638.
8. Hsieh, C.P.; Sun, S.; Kriman, S.; Acharya, S.; Rekesh, D.; Jia, F.; Zhang, Y.; Ginsburg, B. RULER: What's the Real Context Size of Your Long-Context Language Models?, 2024, [arXiv:cs.CL/2404.06654]. Conference on Language Modeling (COLM 2024), https://doi.org/10.48550/arXiv.2404.06654.
9. Teufel, S.; Moens, M. Summarizing Scientific Articles: Experiments with Relevance and Rhetorical Status. *Computational Linguistics* **2002**, *28*, 409–445. https://doi.org/10.1162/089120102762671936.
10. Liakata, M.; Teufel, S.; Siddharthan, A.; Batchelor, C. Corpora for the Conceptualisation and Zoning of Scientific Papers. In Proceedings of the Proceedings of the Seventh International Conference on Language Resources and Evaluation (LREC'10), Valletta, Malta, 2010.
11. Fisas, B.; Ronzano, F.; Saggion, H. A Multi-Layered Annotated Corpus of Scientific Papers. In Proceedings of the Proceedings of the Tenth International Conference on Language Resources and Evaluation (LREC'16), Portorož, Slovenia, 2016; pp. 3081–3088.
12. Dernoncourt, F.; Lee, J.Y. PubMed 200k RCT: A Dataset for Sequential Sentence Classification in Medical Abstracts. In Proceedings of the Proceedings of the Eighth International Joint Conference on Natural Language Processing (Volume 2: Short Papers), Taipei, Taiwan, 2017; pp. 308–313.
13. Cohan, A.; Beltagy, I.; King, D.; Dalvi, B.; Weld, D. Pretrained Language Models for Sequential Sentence Classification. In Proceedings of the Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), Hong Kong, China, 2019; pp. 3693–3699. https://doi.org/10.18653/v1/D19-1383.
14. Dayrell, C.; Candido Jr., A.; Lima, G.; Machado Jr., D.; Copestake, A.; Feltrim, V.; Tagnin, S.; Aluisio, S. Rhetorical Move Detection in English Abstracts: Multi-label Sentence Classifiers and their Annotated Corpora. In Proceedings of the Proceedings of the Eighth International Conference on Language Resources and Evaluation (LREC'12), Istanbul, Turkey, 2012; pp. 1604–1609.
15. Stead, C.; Smith, S.; Busch, P.; Vatanasakdakul, S. Emerald 110k: A Multidisciplinary Dataset for Abstract Sentence Classification. In Proceedings of the Proceedings of the 17th Annual Workshop of the Australasian Language Technology Association, Sydney, Australia, 2019; pp. 120–125.
16. Bolaños-Burgos, F.; Salatino, A.A.; Osborne, F.; Motta, E. Modelling and Classifying the Components of Literature Reviews: A Novel Annotation Schema and Evaluation of Transformer Models. *PeerJ Computer Science* **2026**, *12*, e3921. https://doi.org/10.7717/peerj-cs.3921.
17. Jin, D.; Szolovits, P. Hierarchical Neural Networks for Sequential Sentence Classification in Medical Scientific Abstracts. In Proceedings of the Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP), Brussels, Belgium, 2018; pp. 3100–3109. https://doi.org/10.18653/v1/D18-1349.
18. Brack, A.; Entrup, E.; Stamatakis, M.; Buschermöhle, P.; Hoppe, A.; Ewerth, R. Sequential Sentence Classification in Research Papers Using Cross-Domain Multi-Task Learning. *International Journal on Digital Libraries* **2024**, *25*, 377–400. https://doi.org/10.1007/s00799-023-00392-z.
19. Lan, M.; Zheng, L.; Ming, S.; Kilicoglu, H. Multi-label Sequential Sentence Classification via Large Language Model. In Proceedings of the Findings of the Association for Computational Linguistics: EMNLP 2024. Association for Computational Linguistics, 2024, pp. 16086–16104. https://doi.org/10.18653/v1/2024.findings-emnlp.944.
20. Zhang, Y.; Zhang, C. Extracting Problem and Method Sentence from Scientific Papers: A Context-Enhanced Transformer Using Formulaic Expression Desensitization. *Scientometrics* **2024**, *129*, 3433–3468. https://doi.org/10.1007/s11192-024-05048-6.
21. Hu, Y.; Wan, X. Mining and Analyzing the Future Works in Scientific Articles, 2015, [arXiv:cs.DL/1507.02140]. https://doi.org/10.48550/arXiv.1507.02140.
22. Hao, W.; Li, Z.; Qian, Y.; Wang, Y.; Zhang, C. The ACL FWS-RC: A Dataset for Recognition and Classification of Sentence about Future Works. In Proceedings of the Proceedings of the ACM/IEEE Joint Conference on Digital Libraries (JCDL), Virtual Event, China, 2020; pp. 261–269. https://doi.org/10.1145/3383583.3398526.
23. Suray, J.; Klemmer, J.H.; Schmüser, J.; Fahl, S. How the Future Works at SOUPS: Analyzing Future Work Statements and Their Impact on Usable Security and Privacy Research. *arXiv preprint arXiv:2405.20785* **2024**.

24. Thaepprasit, T.; Kehasukcharoen, P.; Wang, D.; Denton, R.; Vateekul, P.; Lertvittayakumjorn, P. What Limits Us? Analyzing Self-Reported Limitations in NLP Research. In Proceedings of the Findings of the Association for Computational Linguistics: EMNLP 2026. Association for Computational Linguistics, 2026, pp. 51260–51297. https://doi.org/10.18653/2026.findings-emnlp.2386.
25. Si, C.; Yang, D.; Hashimoto, T. Can LLMs Generate Novel Research Ideas? A Large-Scale Human Study with 100+ NLP Researchers. In Proceedings of the International Conference on Learning Representations, 2025.
26. Al Azher, I.; Mokarrama, M.J.; Guo, Z.; Ray Choudhury, S.; Alhoori, H. BAGELS: Benchmarking the Automated Generation and Extraction of Limitations from Scholarly Text. In Proceedings of the Findings of the Association for Computational Linguistics: EMNLP 2025. Association for Computational Linguistics, 2025, pp. 19279–19294. https://doi.org/10.18653/v1/2025.findings-emnlp.1050.
27. Al Azher, I.; Mokarrama, M.J.; Guo, Z.; Ray Choudhury, S.; Alhoori, H. FutureGen: A RAG-based Approach to Generate the Future Work of Scientific Article. In Proceedings of the 2025 IEEE International Conference on eScience (eScience). IEEE, 2025, pp. 427–438. https://doi.org/10.1109/eScience65000.2025.00087.
28. Cheng, J.; Liu, X.; Tao, Z.; Gu, H.; Wang, Y. Research and Application of Future Work Sentence Extraction Methods Based on Large Language Models: Taking Papers in the Natural Language Processing (NLP) Field as Example. *Science Focus* **2026**, *21*, 81–98. In Chinese; English title and abstract supplied by the publisher, https://doi.org/10.15978/j.cnki.1673-5668.20250721.
29. Salem, N.M.; White, E.; Bada, M.; Hunter, L. GAPMAP: Mapping Scientific Knowledge Gaps in Biomedical Literature Using Large Language Models. *arXiv preprint arXiv:2510.25055* **2025**.
30. Faizullah, A.R.B.M.; Urlana, A.; Mishra, R. LimGen: Probing the LLMs for Generating Suggestive Limitations of Research Papers. In Proceedings of the Machine Learning and Knowledge Discovery in Databases. Research Track (ECML PKDD 2024), Vilnius, Lithuania, 2024; Vol. 14942, *Lecture Notes in Computer Science*, pp. 106–124. https://doi.org/10.1007/978-3-031-70344-7_7.
31. Yin, W.; Hay, J.; Roth, D. Benchmarking Zero-shot Text Classification: Datasets, Evaluation and Entailment Approach. In Proceedings of the Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP). Association for Computational Linguistics, 2019, pp. 3914–3923. https://doi.org/10.18653/v1/D19-1404.
32. Ma, Y.; Cao, Y.; Hong, Y.; Sun, A. Large Language Model Is Not a Good Few-shot Information Extractor, but a Good Reranker for Hard Samples! In Proceedings of the Findings of the Association for Computational Linguistics: EMNLP 2023, Singapore, 2023; pp. 10572–10601. https://doi.org/10.18653/v1/2023.findings-emnlp.710.
33. Chen, L.; Zaharia, M.; Zou, J. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance, 2023, [arXiv:cs.LG/2305.05176]. https://doi.org/10.48550/arXiv.2305.05176.
34. Yu, T.; Xu, A.; Akkiraju, R. In Defense of RAG in the Era of Long-Context Language Models, 2024, [arXiv:cs.CL/2409.01666]. https://doi.org/10.48550/arXiv.2409.01666.
35. Wu, Y.; Iso, H.; Pezeshkpour, P.; Bhutani, N.; Hruschka, E. Less is More for Long Document Summary Evaluation by LLMs. In Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 2: Short Papers), St. Julian's, Malta, 2024; pp. 330–343. https://doi.org/10.18653/v1/2024.eacl-short.29.
36. Lan, M. SAL_Type_Classification: Data and Code for Self-Acknowledged Limitation Categorization. GitHub repository, 2024.
37. Xiang, Y. FWS: Future Work Sentence Recognition and Classification Data. GitHub repository, recognition corpus file, 2022.
38. Cohen, J. A Coefficient of Agreement for Nominal Scales. *Educational and Psychological Measurement* **1960**, *20*, 37–46. https://doi.org/10.1177/001316446002000104.
39. Warner, B.; Chaffin, A.; Clavié, B.; Weller, O.; Hallström, O.; Taghadouini, S.; Gallagher, A.; Biswas, R.; Ladhak, F.; Aarsen, T.; et al. Smarter, Better, Faster, Longer: A Modern Bidirectional Encoder for Fast, Memory Efficient, and Long Context Finetuning and Inference. In Proceedings of the Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Association for Computational Linguistics, 2025, pp. 2526–2547. https://doi.org/10.18653/v1/2025.acl-long.127.
40. tasksource. ModernBERT-large-nli Model Card. Hugging Face model, 2025. Revision `ca476cb9`.
41. Sileo, D. tasksource: A Large Collection of NLP Tasks with a Structured Dataset Preprocessing Framework. In Proceedings of the Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024), 2024, pp. 15655–15684.
42. Kwon, W.; Li, Z.; Zhuang, S.; Sheng, Y.; Zheng, L.; Yu, C.H.; Gonzalez, J.E.; Zhang, H.; Stoica, I. Efficient Memory Management for Large Language Model Serving with PagedAttention. In Proceedings of the Proceedings of the 29th Symposium on Operating Systems Principles (SOSP), Koblenz, Germany, 2023; pp. 611–626. https://doi.org/10.1145/3600006.3613165.
43. Gu, Y.; Tinn, R.; Cheng, H.; Lucas, M.; Usuyama, N.; Liu, X.; Naumann, T.; Gao, J.; Poon, H. Domain-Specific Language Model Pretraining for Biomedical Natural Language Processing. *ACM Transactions on Computing for Healthcare* **2021**, *3*, 1–23. https://doi.org/10.1145/3458754.
44. Beltagy, I.; Lo, K.; Cohan, A. SciBERT: A Pretrained Language Model for Scientific Text. In Proceedings of the Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing. Association for Computational Linguistics, 2019, pp. 3615–3620. https://doi.org/10.18653/v1/D19-1371.

45. Sanh, V.; Debut, L.; Chaumond, J.; Wolf, T. DistilBERT, a Distilled Version of BERT: Smaller, Faster, Cheaper and Lighter. *arXiv preprint arXiv:1910.01108* **2019**.
46. Reimers, N.; Gurevych, I. Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks. In Proceedings of the Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing. Association for Computational Linguistics, 2019, pp. 3982–3992. https://doi.org/10.18653/v1/D19-1410.
47. Xiao, S.; Liu, Z.; Zhang, P.; Muennighoff, N.; Lian, D.; Nie, J.Y. C-Pack: Packed Resources For General Chinese Embeddings. In Proceedings of the Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval. Association for Computing Machinery, 2024, pp. 641–649. https://doi.org/10.1145/3626772.3657878.
48. Wang, L.; Yang, N.; Huang, X.; Jiao, B.; Yang, L.; Jiang, D.; Majumder, R.; Wei, F. Text Embeddings by Weakly-Supervised Contrastive Pre-training, 2022, [2212.03533]. https://doi.org/10.48550/arXiv.2212.03533.
49. Efron, B.; Tibshirani, R.J. *An Introduction to the Bootstrap*; Chapman and Hall/CRC: New York, 1993. https://doi.org/10.1201/9780429246593.
50. Alnakri, Y.; Orabi, O.; Mohammed, R.; Alnoukari, M. LR-AI: Evaluating an End-to-End LLM Pipeline from Search to Reading List for Systematic Reviews, 2026. https://doi.org/10.5281/zenodo.24018390.
51. Alnakri, Y.; Orabi, O.; Aldaghstany, H.S. Gap2Idea: Code for Extracting Author-Stated Research Gaps. GitHub repository, 2026. Tag `preprint-v1`.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.